Problem Set for Strength of Materials

Edited by James Lord and Jacob Grohs

2024-08-27

Table of contents

Welcome to the Problem Set for Strength of Materials

How to Use This Resource

Note: This version of the problem set is a review copy for in-class piloting. This is not the final, published version of the problem set.

The navigation bar of this site will allow you to browse to the specific problems you want to view and solve. The Problem Submission App enables you to create your own unique problem used to solve homework problems. Once your problem is generated, you will be able to submit an answer to the problem and receive instantaneous right/wrong feedback before attempting another solution. At the end of your work, you can download a file log of your submissions to share with your instructor. The problem generation process is based on the ID number you use and so if you attempt the problem from a different computer or at a different time, the variables generated for you will be stable across these runs. However, the log downloader is specific to your current session and so attempting the problem across devices does not save attempt across devices

Additionally, there are versions of the problems with fixed variables that do not use the Problem Submission App. These are available to you also in the sidebar should you want a more traditional textbook listing of problems.

If you have any questions or issues while using this system, you are encouraged to contact your instructor and they will help work through the issues with the developers.

Copyright Information

Note: This version of the problem set is being slightly modified by Greg Teichert so that the correct answer is not given to the student and the problem interface is simplified, removing the ID input (a fixed value is used) and option to download, and addition to corrections to problems.

ACKNOWLEDGMENTS

The team gratefully acknowledges the generosity of Kurt Gramoll. Many of the problems were adapted from Kurt Gramoll's collection in the Apple Store and Google Play and released with permission under a CC BY NC SA 4.0 license.

Problem Set Editor and Content Coordinator

James K. Lord, Associate Collegiate Professor, Virginia Tech

Authors of Problems

Kurt Gramoll, Emeritus Professor, University of Oklahoma

James K. Lord, Mechanical Engineering, Virginia Tech

Chris Galitz, Mechanical Engineering, Virginia Tech

Frances Davis, Department of Engineering, Brightpoint Community College

Sneha Davison. Mechanical Engineering, Virginia Tech

Richard Kent, Department of Mechanical & Aerospace Engineering, University of Virginia

Nina Lord, Department of Engineering, North Virginia Community College

Amy Richardson, Dept of Engineering, North Virginia Community College / Engineering Education, Virginia Tech

Joao Soares, Department of Mechanical and Nuclear Engineering, Virginia Commonwealth University

Sevak Tahmasian, Mechanical Engineering, Virginia Tech

Figure Design of Problems

Kindred Grey, Graphic Design and Production Specialist, University Libraries, Virginia Tech

Software Editor

Jacob Grohs, Associate Professor, Engineering Education, Virginia Tech

Review, Problem Redevelopment, and Programming

James K. Lord, Associate Collegiate Professor, Mechanical Engineering, Virginia Tech

Olivia Ryan, Doctoral Student, Engineering Education, Virginia Tech

Benjamin E. Chaback, Doctoral Student, Engineering Education, Virginia Tech

Layout and Design

Kindred Grey, Graphic Design and Production Specialist, University Libraries, Virginia Tech

Project Coordination and Management

Anita Walz, Associate Professor, University Libraries, Virginia Tech

FUNDING

This project was made possible in part through funding from an Open Course Grant from VIVA (The Virtual Library of Virginia) and the Open Education Initiative of the University Libraries at Virginia Tech.

Suggested citation: James Lord and Jacob Grohs, eds. (2024). Problem Set for Strength of Materials. Blacksburg: Virginia Tech Publishing. https://engineeringmechanicsoer.github.io/StrengthOfMaterialsExercises. Licensed with CC BY NC-SA 4.0 https://creativecommons.org/licenses/by-nc-sa/4.0.

Publisher: This work is published by the Department of Mechanical Engineering and Department of Engineering Education at Virginia Tech in association with the Open Education Initiative and Virginia Tech Publishing.

Virginia Tech Department of Mechanical Engineering

445 Goodwin Hall, 635 Prices Fork Road

Blacksburg„ VA 24061 USA

https://me.vt.edu

Department of Engineering Education at Virginia Tech

345 Goodwin Hall, 635 Prices Fork Rd,
Blacksburg, VA 24061 USA
https://enge.vt.edu

Open Education Initiative
University Libraries at Virginia Tech
560 Drillfield Drive Blacksburg, VA 24061 USA
openeducation@vt.edu

Virginia Tech Publishing
University Libraries at Virginia Tech
560 Drillfield Drive Blacksburg, VA 24061 USA
https://publishing.vt.edu | publishing@vt.edu

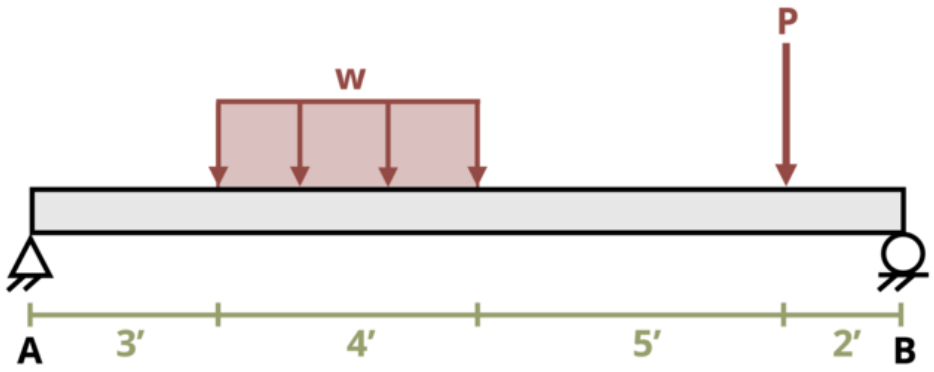
Part I

Problem Submission App

Part II

Chapter 1 Problems

Problem 1.1 - Equilibrium in 2D



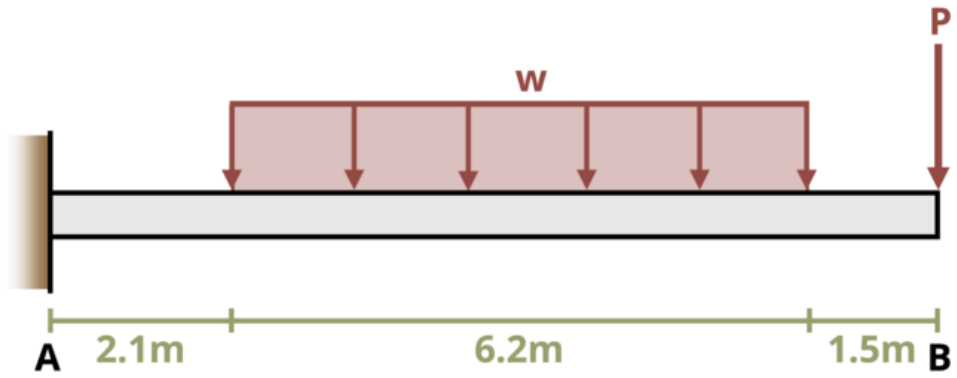
[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "641"
P = reactive.Value("__")
w = reactive.Value("__")
```

```
attempts = ["Timestamp,Attempt,Answer1,Answer2,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kip", placeholder="Please enter your
    ui.input_text("answer2", "Your Answer 2 in units of kip", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The simply-supported beam is subjected to the loading shown. Determine the
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        P.set(random.randrange(5,50,1))
        w.set(random.randrange(20,100,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.
```

```
        # Calculate the instructor's answer and determine if the user's answer is correct.
        instr1 = 4*w()+P()-(20*w()+12*P())/14
        instr2 = (20*w()+12*P())/14

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)

        if correct1 and correct2:
            check = f"{'both correct.'}"
        elif correct1:
            check = f" {'correct for answer 1 and incorrect for answer 2.'}"
        elif correct2:
            check = f"{'incorrect for answer 1 and correct for answer 2.'}"
        else:
            check = f"{'both incorrect.'}"

        correct_indicator = "JL" if correct1 and correct2 else "JG"

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

        feedback = ui.markdown(f"Your answers of {input.answer1()} and {input.answer2()} are
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Problem 1.2 - Equilibrium in 2D



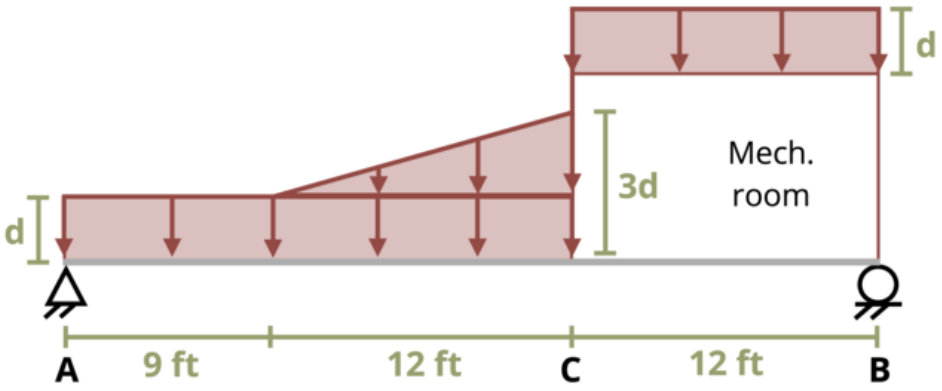
[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "642"
P = reactive.Value("__")
w = reactive.Value("__")
```

```
attempts = ["Timestamp,Attempt,Answer1,Answer2,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kN", placeholder="Please enter your a
    ui.input_text("answer2", "Your Answer 2 in units of kN-m", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The cantilever beam is subjected to the loading shown. Determine the reacti
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        P.set(random.randrange(5,30,1))
        w.set(random.randrange(20,50,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.
```

```
        # Calculate the instructor's answer and determine if the user's answer is correct.
        instr1 = P()+w()*6.2
        instr2 = P()*(2.1+6.2+1.5)+w()*6.2*5.2

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)

        if correct1 and correct2:
            check = f"{'both correct.'}"
        elif correct1:
            check = f" {'correct for answer 1 and incorrect for answer 2.'}"
        elif correct2:
            check = f"{'incorrect for answer 1 and correct for answer 2.'}"
        else:
            check = f"{'both incorrect.'}"

        correct_indicator = "JL" if correct1 and correct2 else "JG"

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

        feedback = ui.markdown(f"Your answers of {input.answer1()} and {input.answer2()} are
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Problem 1.3 - Equilibrium in 2D



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

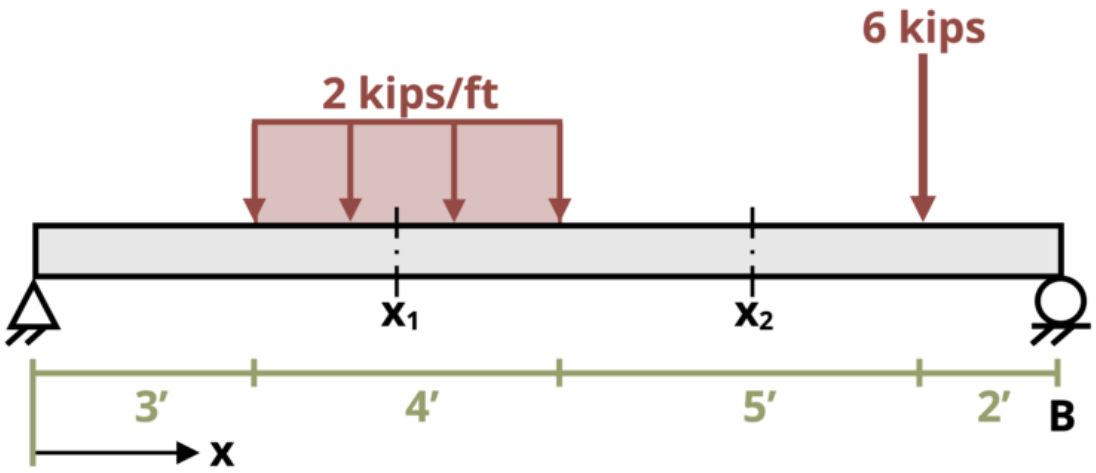
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "643"
d = reactive.Value("__")
w = reactive.Value("__")
```

```
attempts = ["Timestamp,Attempt,Answer1,Answer2,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kip", placeholder="Please enter your
    ui.input_text("answer2", "Your Answer 2 in units of kip", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The simply-supported roof beam AB is subjected to snow loads as a uniform l
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(10,30,1))
        w.set(random.randrange(40,80,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.
```

```
        # Calculate the instructor's answer and determine if the user's answer is correct.
        w1 = w()*12*d()
        w2 = w()*21*d()
        w3 = 0.5*w()*2*d()*12
        instr1 = (w1+w2+w3-(10.5*w2+17*w3+21*w1/2+33*w1/2)/33)/1000
        instr2 = ((10.5*w2+17*w3+21*w1/2+33*w1/2)/33)/1000

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)

        if correct1 and correct2:
            check = f"{'both correct.'}"
        elif correct1:
            check = f" {'correct for answer 1 and incorrect for answer 2.'}"
        elif correct2:
            check = f"{'incorrect for answer 1 and correct for answer 2.'}"
        else:
            check = f"{'both incorrect.'}"

        correct_indicator = "JL" if correct1 and correct2 else "JG"

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

        feedback = ui.markdown(f"Your answers of {input.answer1()} and {input.answer2()} are
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt
```

```
app = App(app_ui, server)
```

Problem 1.4 - Internal Reactions



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

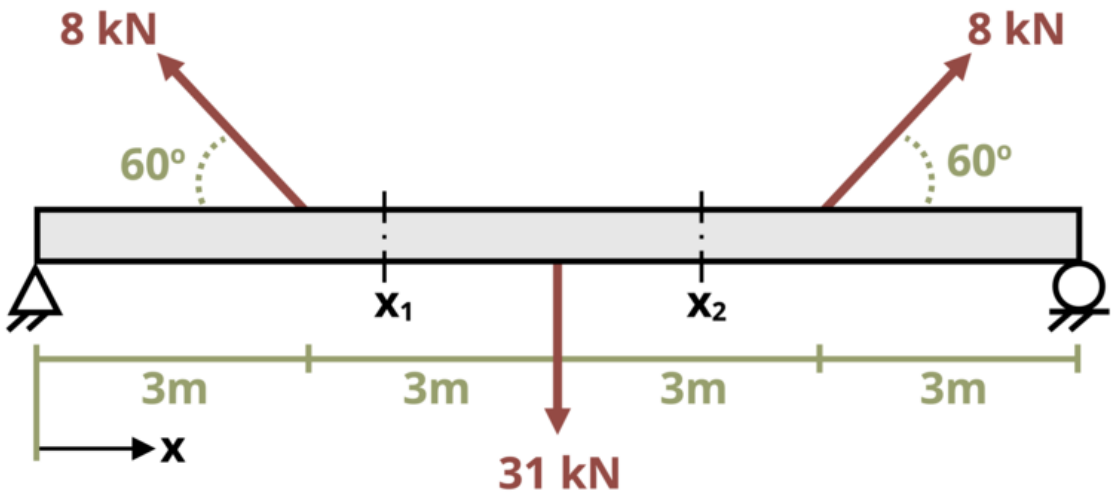
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "644"
```

```
x1 = reactive.Value("__")
x2 = reactive.Value("__")
P = reactive.Value("__")
w = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Answer4,Answer5,Answer6,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kip", placeholder="Please enter your
    ui.input_text("answer2", "Your Answer 2 in units of kip", placeholder="Please enter your
    ui.input_text("answer3", "Your Answer 3 in units of kip-ft", placeholder="Please enter y
    ui.input_text("answer4", "Your Answer 4 in units of kip", placeholder="Please enter your
    ui.input_text("answer5", "Your Answer 5 in units of kip", placeholder="Please enter your
    ui.input_text("answer6", "Your Answer 6 in units of kip-ft", placeholder="Please enter y

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The simply-supported beam is subjected to the loading shown. Determine the
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        x1.set(random.randrange(41,69,1)/10)
```

```
        x2.set(random.randrange(75,115,1)/10)
        P.set(6) #random.randrange(5,25,1))
        w.set(2) #random.randrange(20,50,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        instr1 = 0
        Ay = (P()*2+w()*4*9)/14
        instr2 = Ay-w()*(x1()-3)
        instr3 = w()*(x1()-3)*(3+(x1()-3)/2)+instr2*x1()
        instr4=0
        instr5= Ay-w()*4
        instr6= w()*4*5+instr5*x2()

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
        correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)
        correct4 = math.isclose(float(input.answer4()), instr4, rel_tol=0.01)
        correct5 = math.isclose(float(input.answer5()), instr5, rel_tol=0.01)
        correct6 = math.isclose(float(input.answer6()), instr6, rel_tol=0.01)

        if correct1 and correct2 and correct3 and correct4 and correct5 and correct6:
            check = "All answers are correct."
        else:
            conditions = []
            for i, correct in enumerate([correct1, correct2, correct3, correct4, correct5, c
                if correct:
                    conditions.append(f"correct for answer {i}")
                else:
                    conditions.append(f"incorrect for answer {i}")
            check = "; ".join(conditions) + "."

        correct_indicator = "JL" if correct1 and correct2 and correct3 and correct4 and corre

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
```

```
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

        feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, {inpu
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Problem 1.5 - Internal Reactions



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

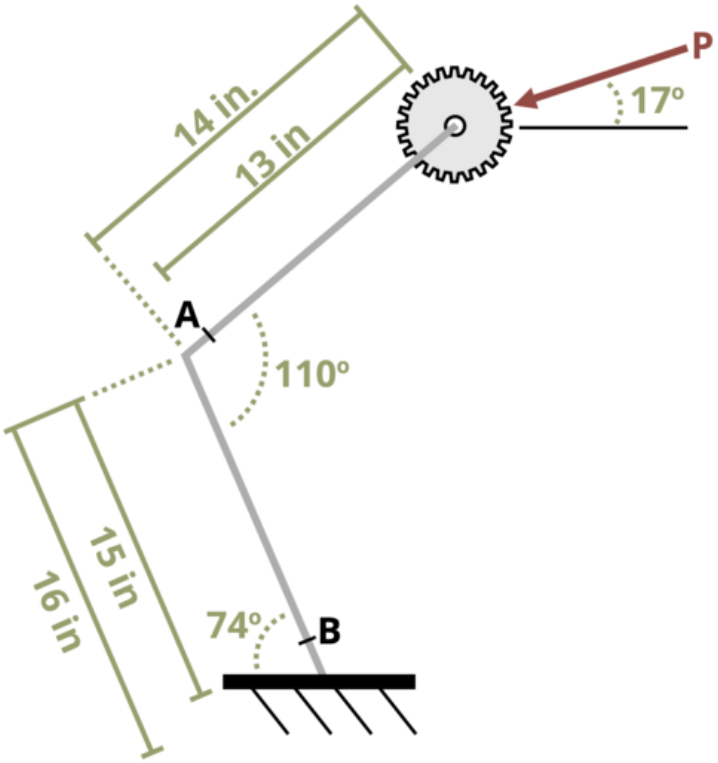
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```
problem_ID = "645"
x = reactive.Value("__")
F1 = reactive.Value("__")
F2 = reactive.Value("__")
F3 = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kN", placeholder="Please enter your a
    ui.input_text("answer2", "Your Answer 2 in units of kN", placeholder="Please enter your a
    ui.input_text("answer3", "Your Answer 3 in units of kN-m", placeholder="Please enter your

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"The simply-supported beam is subjected to the loading shown. Determine the
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        x.set(6.8) #random.randrange(61,89,1)/10)
        F1.set(8) #random.randrange(5,15,1))
        F2.set(31) #random.randrange(20,40,1))
```

```
        F3.set(8) #random.randrange(5,15,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        Ax = F1()*math.cos(math.pi/3)-F3()*math.cos(math.pi/3)
        Ay = (-F1()*math.sin(math.pi/3)*9-F3()*math.sin(math.pi/3)*3+F2()*6)/12
        instr1 = F1()*math.cos(math.pi/3)
        instr2 = Ay+F1()*math.sin(math.pi/3)-31
        instr3 = -F1()*math.sin(math.pi/3)*3+instr2*x()+31*6

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
        correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)

        if correct1 and correct2 and correct3:
            check = "All answers are correct."
        else:
            conditions = []
            for i, correct in enumerate([correct1, correct2, correct3], start=1):
                if correct:
                    conditions.append(f"correct for answer {i}")
                else:
                    conditions.append(f"incorrect for answer {i}")
            check = "; ".join(conditions) + "."

        correct_indicator = "JL" if correct1 and correct2 and correct3 else "JG"

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

        feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, and {
```

```
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Problem 1.6 - Internal Reactions



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

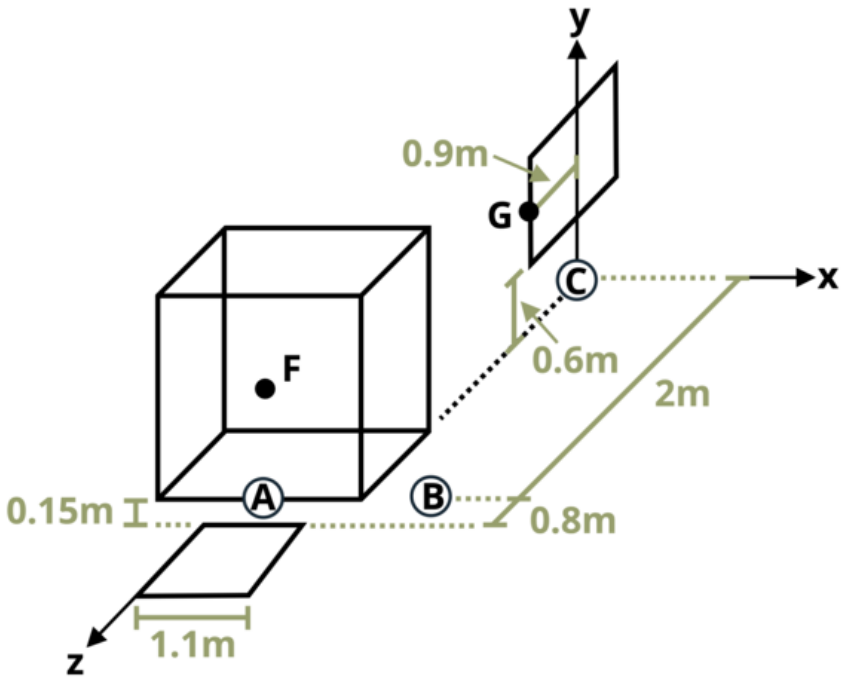
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
```

```
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "646"
P = reactive.Value("__")
theta = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of lb", placeholder="Please enter your a
    ui.input_text("answer2", "Your Answer 2 in units of lb", placeholder="Please enter your a
    ui.input_text("answer3", "Your Answer 3 in units of lb-in", placeholder="Please enter you

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A gear shift, once engaged, can be modeled as the bent cantilever beam sho
            )
        ]

    #@reactive.Effect
```

```
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        P.set(random.randrange(20,75,1))
        theta.set(random.randrange(10,25,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        instr1 = abs(P()*math.cos((70+theta())*math.pi/180))
        instr2 = abs(P()*math.sin((70+theta())*math.pi/180))
        instr3 = abs(P()*math.sin((70+theta())*math.pi/180)*(15+14*math.cos(70*math.pi/180))

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
        correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)

        if correct1 and correct2 and correct3:
            check = "All answers are correct."
        else:
            conditions = []
            for i, correct in enumerate([correct1, correct2, correct3], start=1):
                if correct:
                    conditions.append(f"correct for answer {i}")
                else:
                    conditions.append(f"incorrect for answer {i}")
            check = "; ".join(conditions) + "."

        correct_indicator = "JL" if correct1 and correct2 and correct3 else "JG"

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a
```

```
        feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, and {i
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Problem 1.7 - Equilibrium & Rections in 3D



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

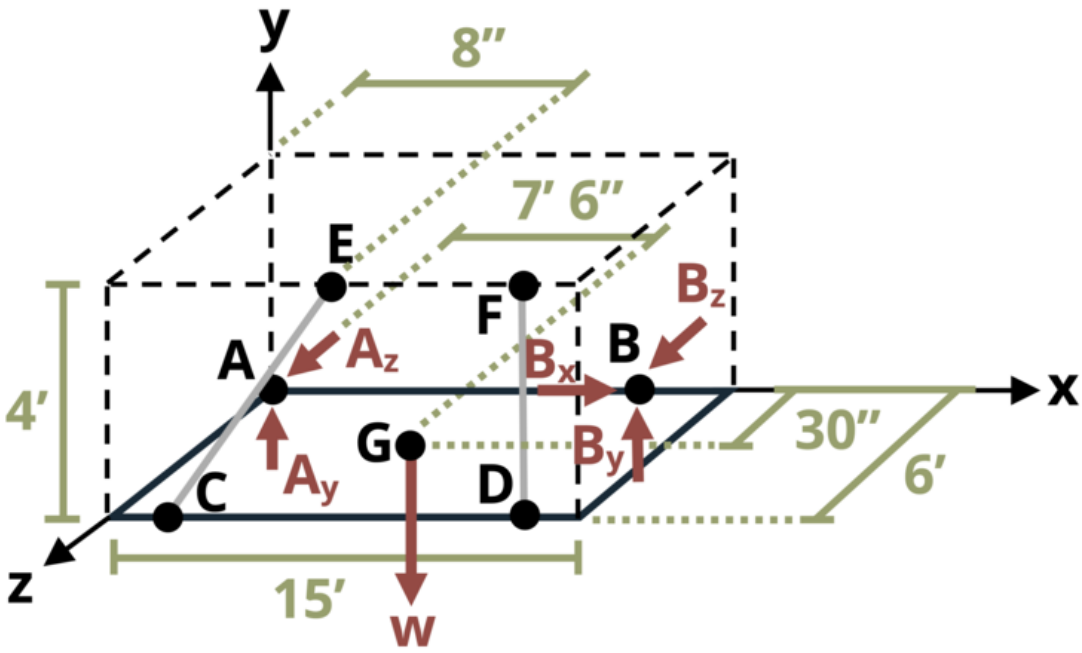
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

```
def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "647"
W = reactive.Value("__")
P = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of kN", placeholder="Please enter your a
    ui.input_text("answer2", "Your Answer 2 in units of kN", placeholder="Please enter your a
    ui.input_text("answer3", "Your Answer 3 in units of kN", placeholder="Please enter your a

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A 3-wheeled forklift weighing W = {W()} kN centered at G is carrying an iml
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        P.set(random.randrange(100,200,1)/10)
```

```
        W.set(random.randrange(10,30,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        instr1 = ((0.6*W()+2.8*P())/2+(0.15*P())/0.55)/2
        instr2 = (0.6*W()+2.8*P())/2-instr1
        instr3 = P()+W()-instr1-instr2

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
        correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)

        if correct1 and correct2 and correct3:
            check = "All answers are correct."
        else:
            conditions = []
            for i, correct in enumerate([correct1, correct2, correct3], start=1):
                if correct:
                    conditions.append(f"correct for answer {i}")
                else:
                    conditions.append(f"incorrect for answer {i}")
            check = "; ".join(conditions) + "."

        correct_indicator = "JL" if correct1 and correct2 and correct3 else "JG"

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

        feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, and {
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)
```

```
    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Problem 1.8 - Equilibrium & Reactions in 3D



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

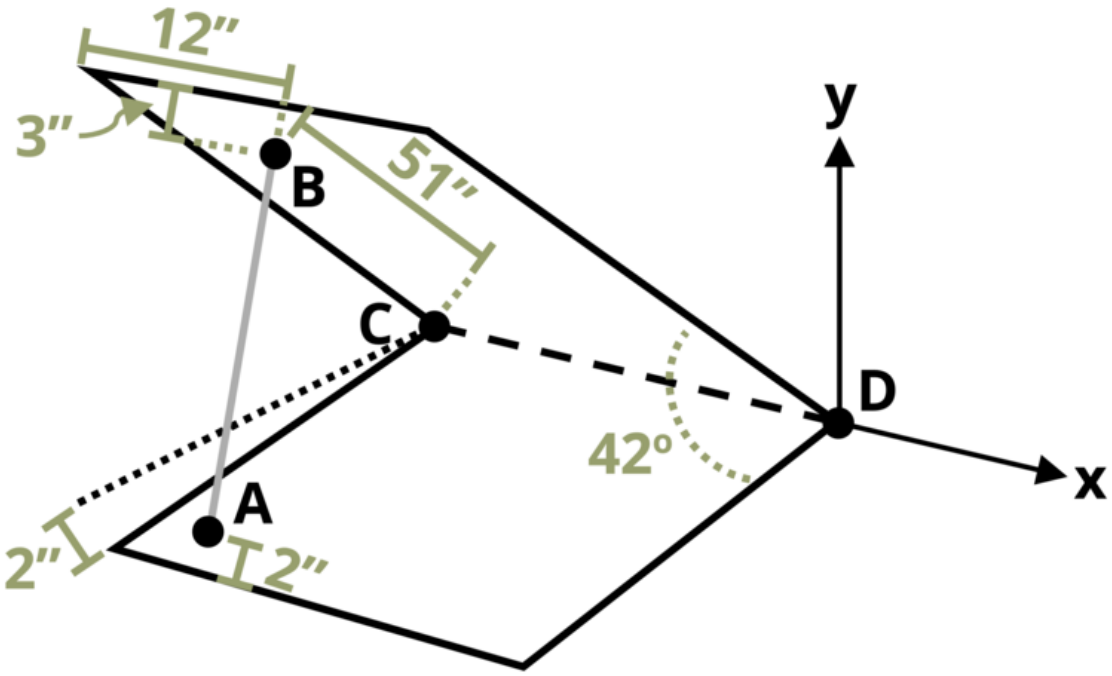
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

```
def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "648"
w = reactive.Value("__")
L = reactive.Value("__")
weight = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Answer4,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of lb", placeholder="Please enter your a
    ui.input_text("answer2", "Your Answer 2 in units of lb", placeholder="Please enter your a
    ui.input_text("answer3", "Your Answer 3 in units of lb", placeholder="Please enter your a
    ui.input_text("answer4", "Your Answer 4 in units of lb", placeholder="Please enter your a

    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"An awning frame of width w = {w()} ft and length L = {L()} ft is attached t
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
```

```
        random.seed(101)
        w.set(random.randrange(3,10,1))
        L.set(w()*random.randrange(20,30,1)/10)
        weight.set(random.randrange(50,100,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        instr1 = 2.5*weight()/(1.106*w())
        instr2 = -0.3869*instr1
        Ay = (L()*0.552*instr1-7.5*weight())/(-L())
        instr3 = weight()-Ay-instr1*(0.552)-instr1*(0.554)
        Az = (L()*w()/7.238*instr1-0.0921*instr1*w()+w()*0.0461*instr1)/L()
        instr4 = -Az+instr1*w()/7.238+instr1*w()/7.22

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
        correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)
        correct4 = math.isclose(float(input.answer4()), instr4, rel_tol=0.01)


        if correct1 and correct2 and correct3 and correct4:
            check = "All answers are correct."
        else:
            conditions = []
            for i, correct in enumerate([correct1, correct2, correct3, correct4], start=1):
                if correct:
                    conditions.append(f"correct for answer {i}")
                else:
                    conditions.append(f"incorrect for answer {i}")
            check = "; ".join(conditions) + "."

        correct_indicator = "JL" if correct1 and correct2 and correct3 and correct4 else "JG

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
```

```
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.an

        feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, {input
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Problem 1.9



[Problem adapted from © Chris Galitz CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

```
def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "649"
w = reactive.Value("__")
L = reactive.Value("__")
weight = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Answer3,Answer4,Answer5,Answer6,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of lb", placeholder="Please enter your a
    ui.input_text("answer2", "Your Answer 2 in units of lb", placeholder="Please enter your a
    ui.input_text("answer3", "Your Answer 3 in units of lb", placeholder="Please enter your a
    ui.input_text("answer4", "Your Answer 4 in units of lb", placeholder="Please enter your a
    ui.input_text("answer5", "Your Answer 5 in units of lb", placeholder="Please enter your a
    ui.input_text("answer6", "Your Answer 6 in units of lb", placeholder="Please enter your a


    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"During engine work, a car's hood of length L = {L()} in. and width w = {w()
            )
        ]
```

```
    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(50,70,1))
        w.set(L()+random.randrange(10,20,1))
        weight.set(random.randrange(30,80,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        RDB = [-w()+12, 34.13, 37.9]
        RDA = [-w()+6, 0, L()-2]
        RDC = [-w(), 0, 0]
        RDG = [-w()/2, L()/2*0.6691, L()/2*0.7431]
        def vector_subtraction(RDB, RDA):
            return [x - y for x, y in zip(RDB, RDA)]
        RAB = vector_subtraction(RDB,RDA)
        def vector_magnitude(RAB):
            return math.sqrt(sum(x ** 2 for x in RAB))
        RABmag = vector_magnitude(RAB)
        FbarAB = [x/RABmag for x in RAB]
        comp1 = RDB[1]*FbarAB[2]-RDB[2]*FbarAB[1]
        comp2 = RDB[2]*FbarAB[0]-RDB[0]*FbarAB[2]
        comp3 = RDB[0]*FbarAB[1]-RDB[1]*FbarAB[0]
        FABCross = [comp1, comp2, comp3]
        Weightj = [0, -weight(), 0]
        comp4 = RDG[1]*Weightj[2]-RDG[2]*Weightj[1]
        comp5 = RDG[2]*Weightj[0]-RDG[0]*Weightj[2]
        comp6 = RDG[0]*Weightj[1]-RDG[1]*Weightj[0]
        RDGCross = [comp4, comp5, comp6]
        instr1 = comp4/comp1
        Cract = [0,1,1]
        comp7 = RDC[1]*Cract[2]-RDC[2]*Cract[1]
        comp8 = RDC[2]*Cract[0]-RDC[0]*Cract[2]
        comp9 = RDC[0]*Cract[1]-RDC[1]*Cract[0]
        RDCCross = [comp7, comp8, comp9]
```

```
        instr2 = comp2*instr1/comp8
        instr3 = (comp3*instr1-comp6)/comp9
        instr4 = -FbarAB[0]*instr1
        instr5 = -FbarAB[1]*instr1+weight()-instr3
        instr6 = -FbarAB[2]-instr2

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)
        correct3 = math.isclose(float(input.answer3()), instr3, rel_tol=0.01)
        correct4 = math.isclose(float(input.answer4()), instr4, rel_tol=0.01)
        correct5 = math.isclose(float(input.answer5()), instr5, rel_tol=0.01)
        correct6 = math.isclose(float(input.answer6()), instr6, rel_tol=0.01)

        if correct1 and correct2 and correct3 and correct4 and correct5 and correct6:
            check = "All answers are correct."
        else:
            conditions = []
            for i, correct in enumerate([correct1, correct2, correct3, correct4, correct5, co
                if correct:
                    conditions.append(f"correct for answer {i}")
                else:
                    conditions.append(f"incorrect for answer {i}")
            check = "; ".join(conditions) + "."

        correct_indicator = "JL" if correct1 and correct2 and correct3 and correct4 and corre

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.an

        feedback = ui.markdown(f"Your answers of {input.answer1()}, {input.answer2()}, {input
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
```

```
        yield "Timestamp,Attempt,Answer1,Answer2,Answer3,Answer4,Answer5,Answer6,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Part III

Chapter 2 Problems

Problem 2.1 - Average Normal Stress

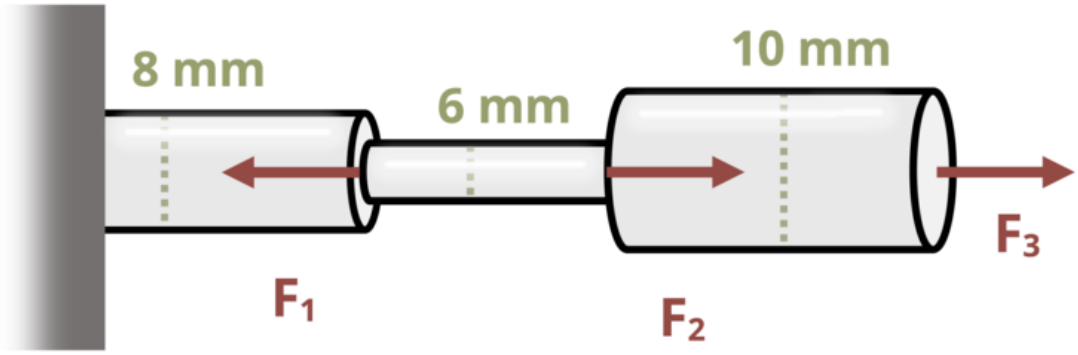


Figure 1: Figure 1: A series of solid circular bars are loaded with three loads

[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

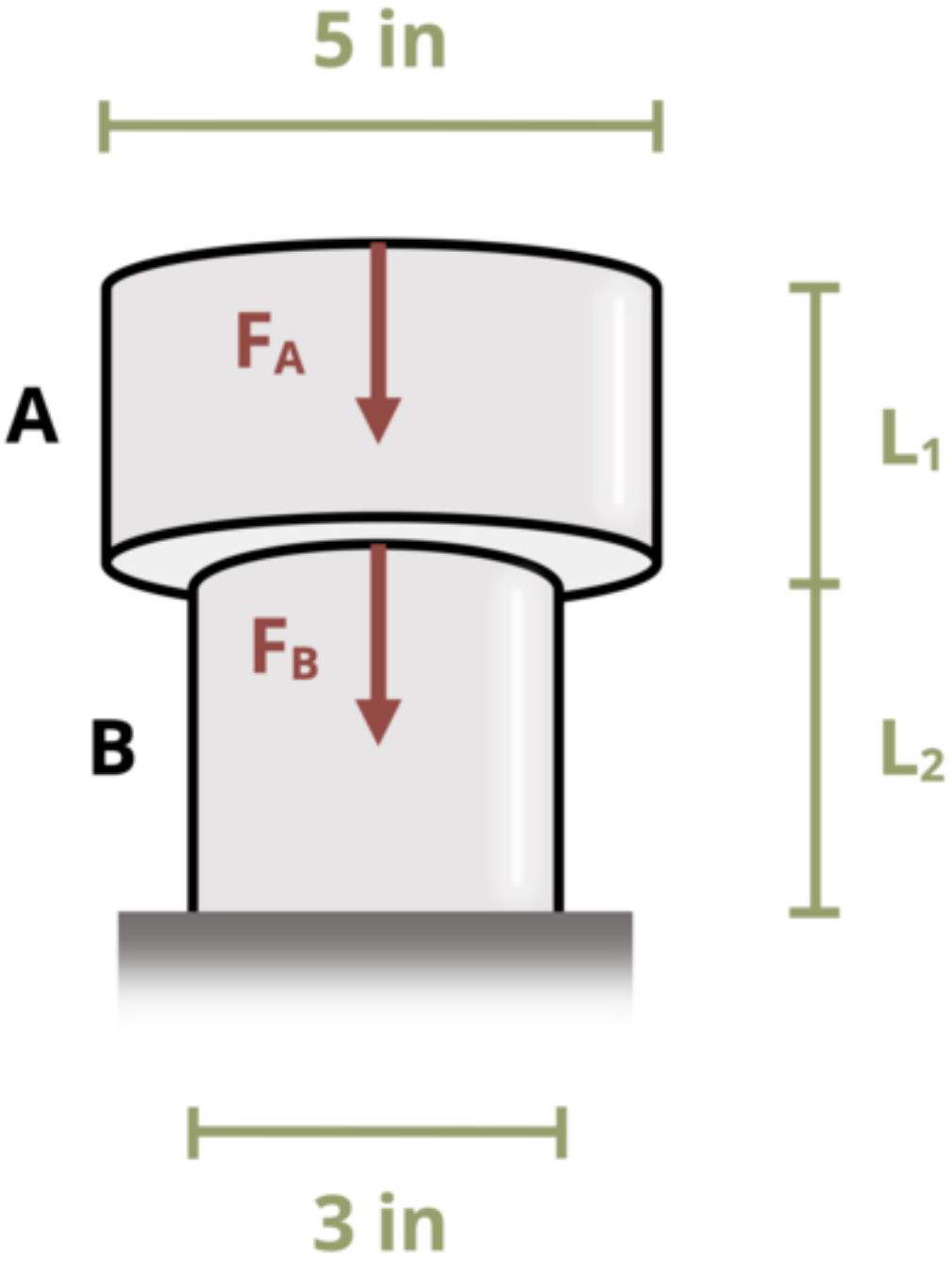
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path


def generate_random_letters(length):
    # Generate a random string of letters of specified length
```

```
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="138"
F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")
d1=8
d2=6
d3=10
attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)


def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A series of solid circular bars are loaded with three loads as s

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(50, 70, 1))
        F2.set(random.randrange(10, 30, 1))
        F3.set(F1()-F2())
```

```
    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s

        # Calculate the instructor's answer and determine if the user's answer is correct.
        instr= (F1()/(math.pi*(d2/(1000*2))**2))/10**6

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
```

```
        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt


# App installation
app = App(app_ui, server)
```

Problem 2.2 - Average Normal Stress

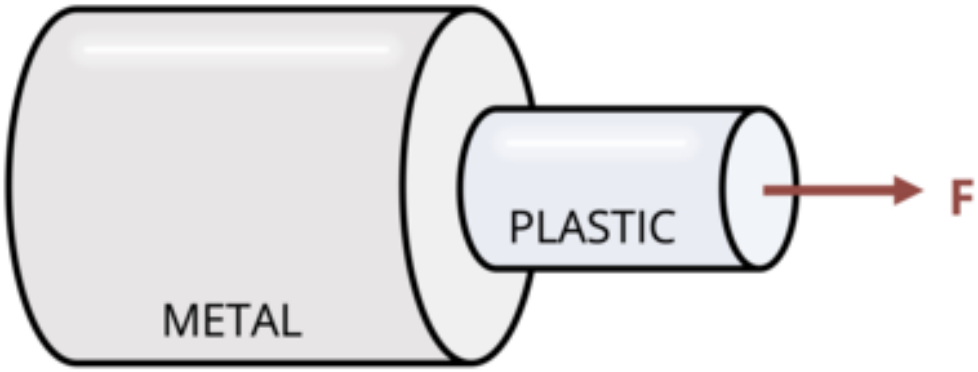


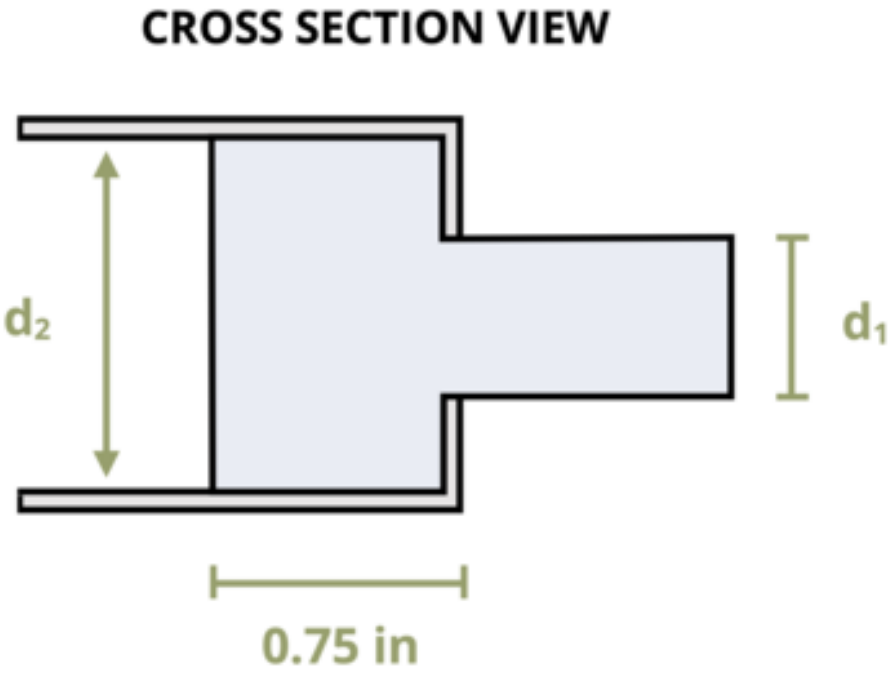
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="139"
L1=reactive.Value("__")
L2=reactive.Value("__")
FA=reactive.Value("__")
FB=reactive.Value("__")
E = 30*10**6

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your ansu
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
```

```
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two cylinders are stacked on top of one another and two forces a

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        FA.set(random.randrange(300, 700, 10))
        FB.set(random.randrange(100, 300, 10))
        L1.set(random.randrange(2, 7, 1))
        L2.set(round(L1() * 1.3,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s

        instr = -(FA()+FB())/(math.pi*1.5**2)
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
```

```
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.3 - Average Normal Stress



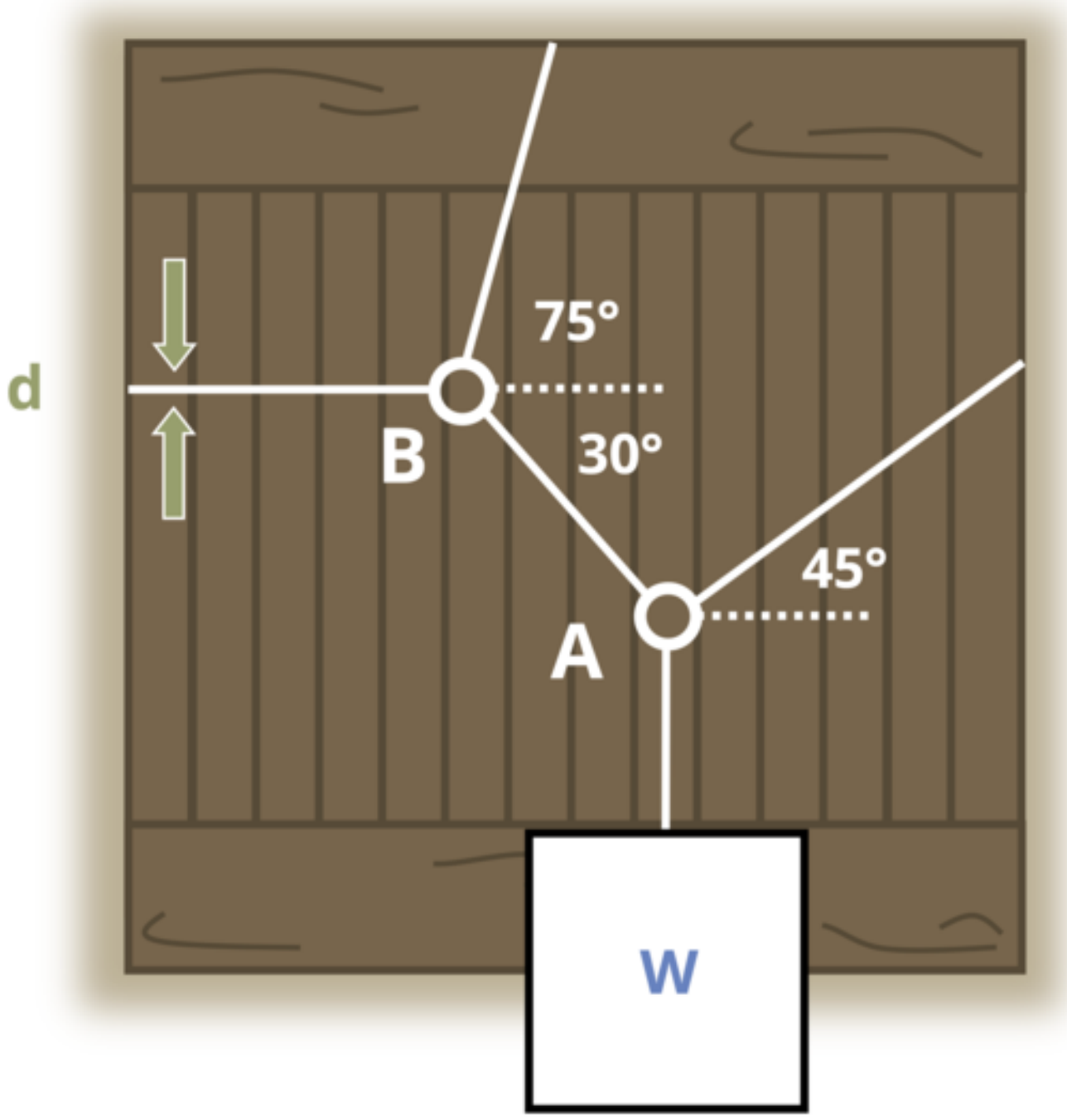


```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="144"
F=reactive.Value("__")
d1=reactive.Value("__")
d2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)


def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
```

```
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A plastic cylindrical peg is constrained by a metal cap as show

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(20, 80, 5))
        d1.set(random.randrange(3, 8, 1)/10)
        d2.set(round(d1() * 1.6, 2))


    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        instr= F()/(math.pi*((d1()/2)**2))
        #check=math.isclose(float(input.answer()),instr,rel_tol=0.001)

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
```

```
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt


# App installation
app = App(app_ui, server)
```

Problem 2.4 - Average Normal Stress

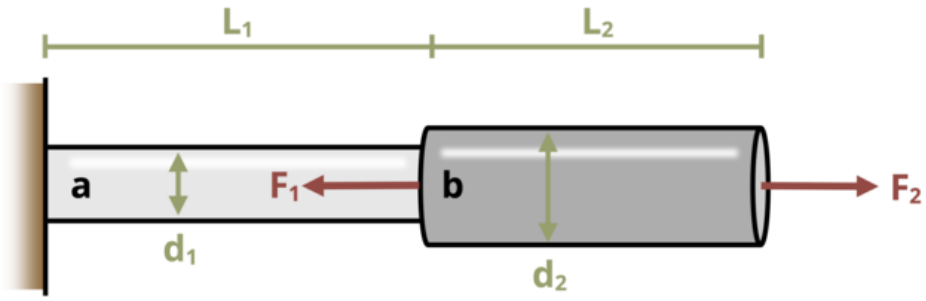


```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path


def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="146"
W=reactive.Value("__")
d=reactive.Value("__")
angle1=math.radians(45)
angle2=math.radians(30)
angle3=math.radians(75)

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ansu
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)


def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)
```

```
@output
@render.ui
def ui_problem_statement():
    randomize_vars()
    return[ui.markdown(f"A crate weighing {W()} kN is suspended by a set of cables. The c

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    W.set(random.randrange(30, 90, 1))
    d.set(random.randrange(20, 60, 1))


@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s

    R1 = W()/(((math.cos(angle1)/math.cos(angle2))*math.sin(angle2))+math.sin(angle1))
    instr = (R1*10**3/(math.pi*((d()/(1000*2))**2)))/10**6
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
```

```
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt


# App installation
app = App(app_ui, server)
```

Problem 2.5 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

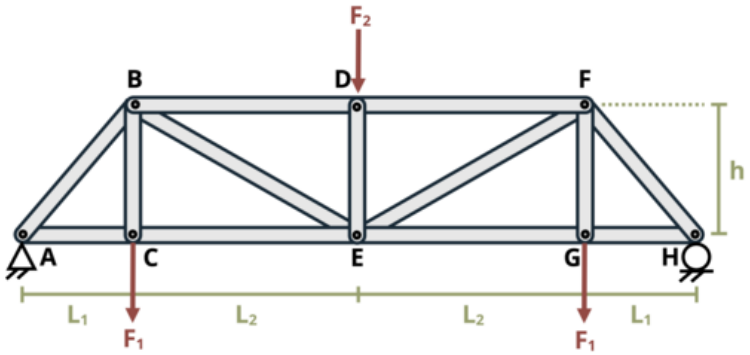
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="141"
F1=reactive.Value("__")
F2=reactive.Value("__")
d1=reactive.Value("__")
d2=reactive.Value("__")
```

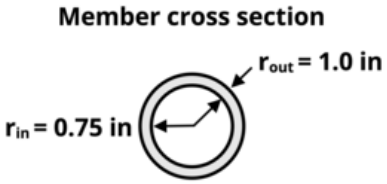
```
attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two circular rods are attached and loaded with forces F<sub>1</s

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10, 50, 1))
        F2.set(random.randrange(10, 50, 1))
        d1.set(random.randrange(10, 40, 1))
        d2.set(d1()*random.randrange(12, 14, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        sa = abs((F2()-F1())/((d1()/2000)**2*math.pi)/1000)
        sb = abs(F2()/((d2()/2000)**2*math.pi)/1000)
        if sa>sb:
            instr = sa
        else:
            instr=sb
```

```
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        # Store the most recent encoded attempt in a reactive value so it persists across su
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writ
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt
```

```
# App installation
app = App(app_ui, server)
```

Problem 2.6 - Average Normal Stress





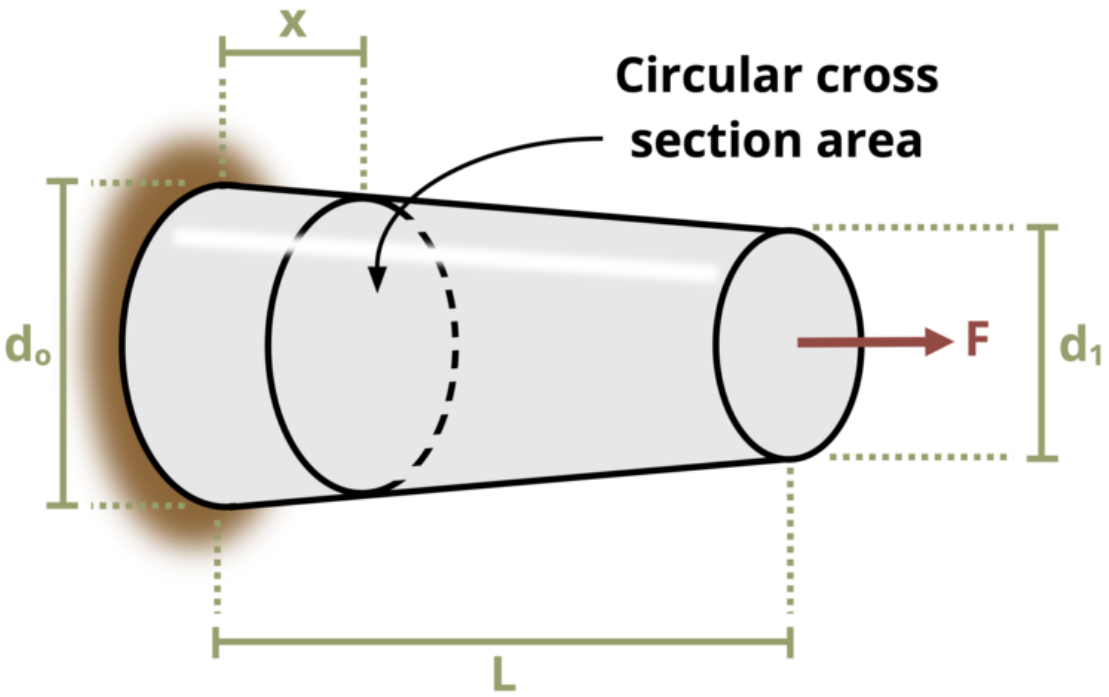
[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="142"
F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")
```

```
attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A truss is subjected to loads F<sub>1</sub> = {F1()} kips, F<sub

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10, 100, 1)/10)
        F2.set(random.randrange(10, 100, 1)/10)
        F3.set(random.randrange(10, 100, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        Hy = (8*F3()+5*F2()+2*F1())/10
        FDF = (-Hy*5+F3()*3)/2
        A = math.pi*(1**2-0.75**2)
        instr = abs(FDF/A)
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
```

```
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.7 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

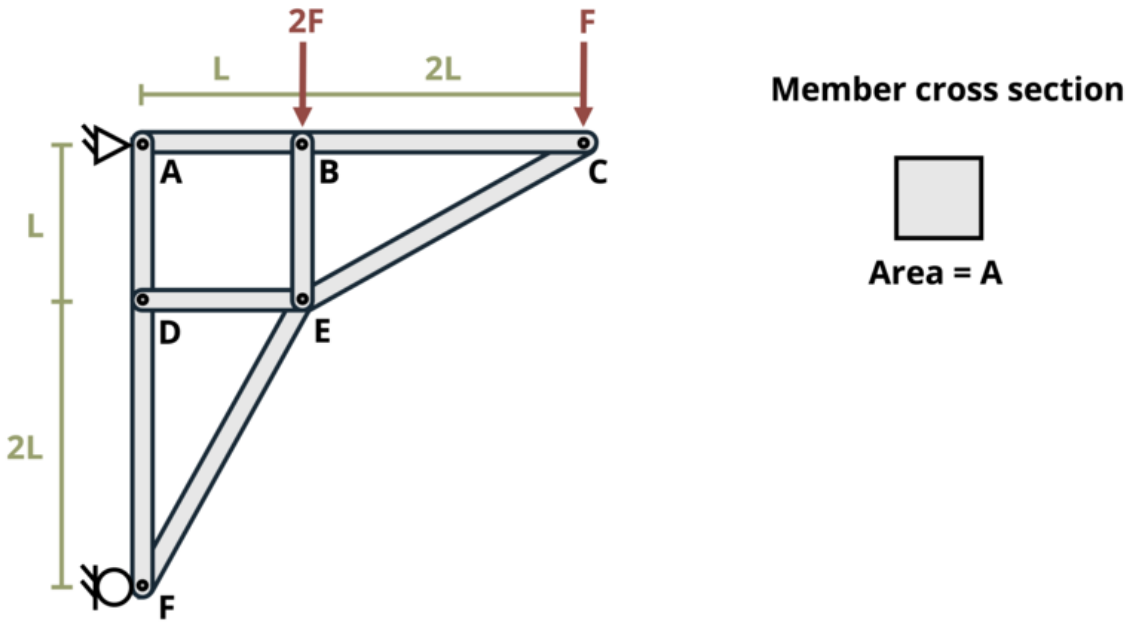
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
```

```
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="143"
F=reactive.Value("__")
L=reactive.Value("__")
d0=reactive.Value("__")
d1=reactive.Value("__")
x=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kPa", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A tapered cylindrical rod is pulled by a force F = {F()} N. If l

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(200, 1000, 10))
        L.set(random.randrange(10, 100, 1)/10)
```

```
        d0.set(random.randrange(300, 700, 10))
        d1.set(d0()/2)
        x.set(L()*random.randrange(2, 4, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        d = d0()/10-(x()/L())*(d0()/10-d1()/10)
        instr = F()/((d/2)**2*math.pi)*10
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
```

```
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.8 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

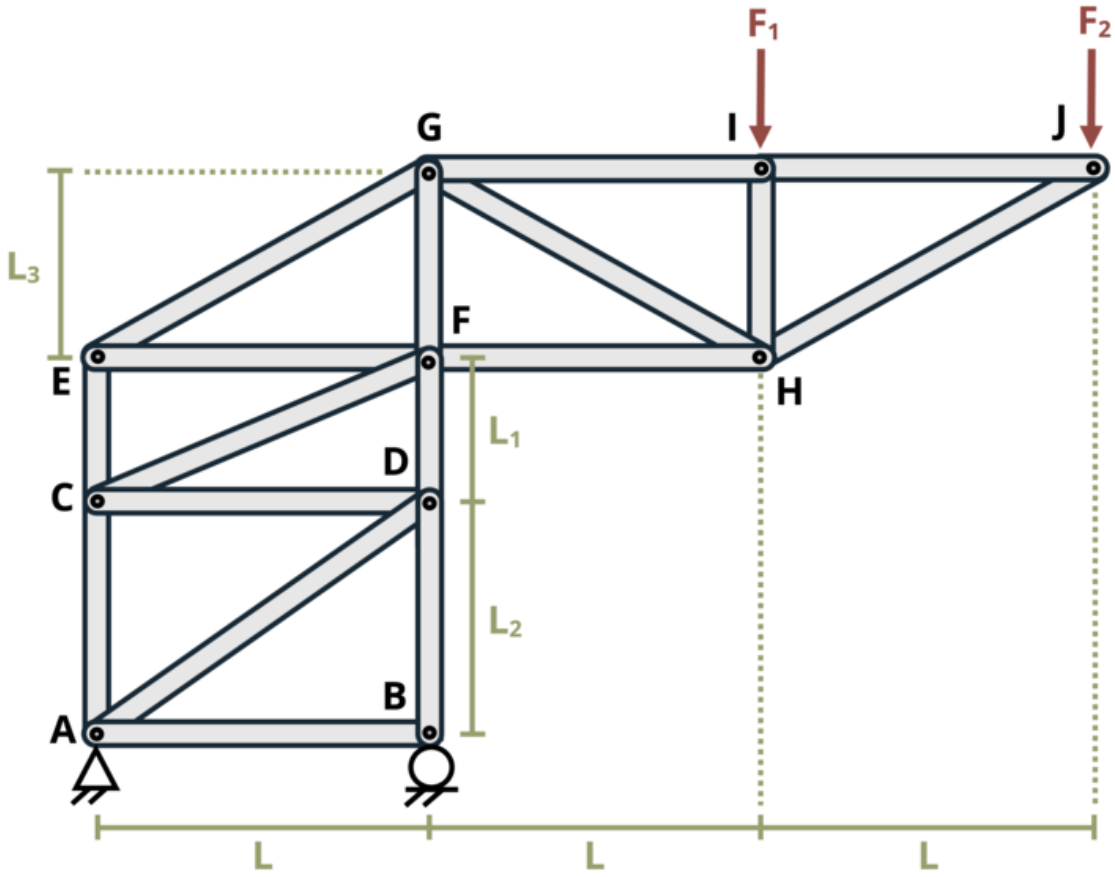
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
```

```
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="149"
A=reactive.Value("__")
F=reactive.Value("__")
L=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A truss is constructed using solid square members of area A = {A

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        A.set(random.randrange(500, 1500, 10))
        F.set(random.randrange(20, 75, 1))
        L.set(random.randrange(5, 10, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
```

```
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        Fx = (2*F()*L()+F()*3*L())/(3*L())
        FEF = -Fx*2.236
        instr = FEF/A()*10**3
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
```

```
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.9 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

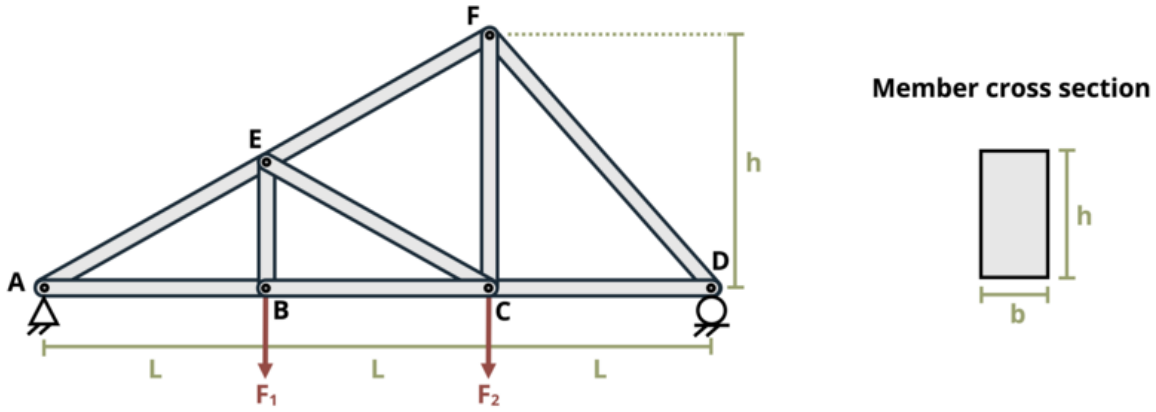
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
```

```
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="150"
A=reactive.Value("__")
F1=reactive.Value("__")
F2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your ansu
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A truss is constructed using members of area A = {A()} in<sup>2<

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
```

```
        random.seed(101)
        A.set(random.randrange(20, 100, 1)/10)
        F1.set(random.randrange(300, 1000, 10))
        F2.set(random.randrange(300, 1000, 10))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        FEG = (F1()*12+F2()*24)/4.615
        instr = FEG/A()
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
```

```
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.10 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

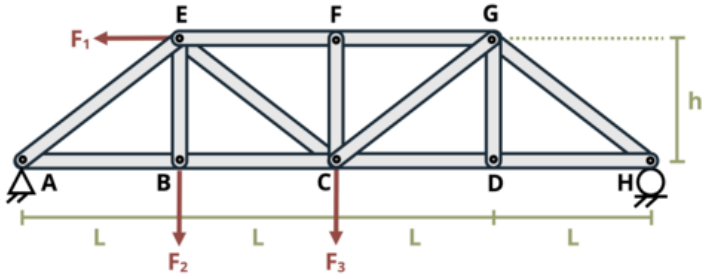
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="152"
b=reactive.Value("__")
h=reactive.Value("__")
```

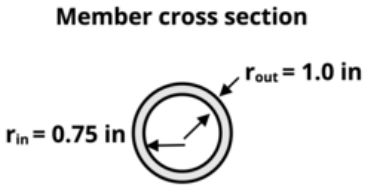
```
F1=reactive.Value("__")
F2=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A truss is constructed of members with rectangular cross-sectio

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        b.set(random.randrange(10, 50, 1)/10)
        h.set(b()*2)
        F1.set(random.randrange(10, 100, 1)/10)
        F2.set(random.randrange(10, 100, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        FD = (6*F2()+3*F1())/9
        FEF = -FD*3/(3*math.sin(63.43*math.pi/180))
        instr = FEF/(b()*h())*1000
```

```
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt
```

```
# App installation
app = App(app_ui, server)
```

Problem 2.11 -Average Normal Stress





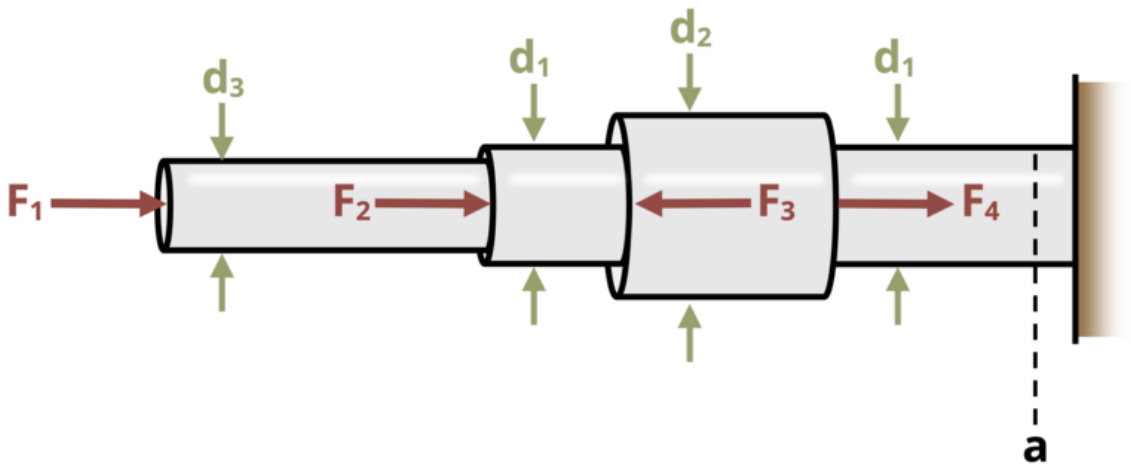
[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="154"
F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]
```

```
app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A truss is constructed of members with the circular cross-sectio

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10, 100, 1)/10)
        F2.set(random.randrange(10, 100, 1)/10)
        F3.set(random.randrange(10, 100, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        Hy = (F3()*8+F2()*4-F1()*4)/16
        FEC = (F3()-Hy)/math.cos(45*math.pi/180)
        instr = FEC/1.374
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"
```

```
        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.12 -Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

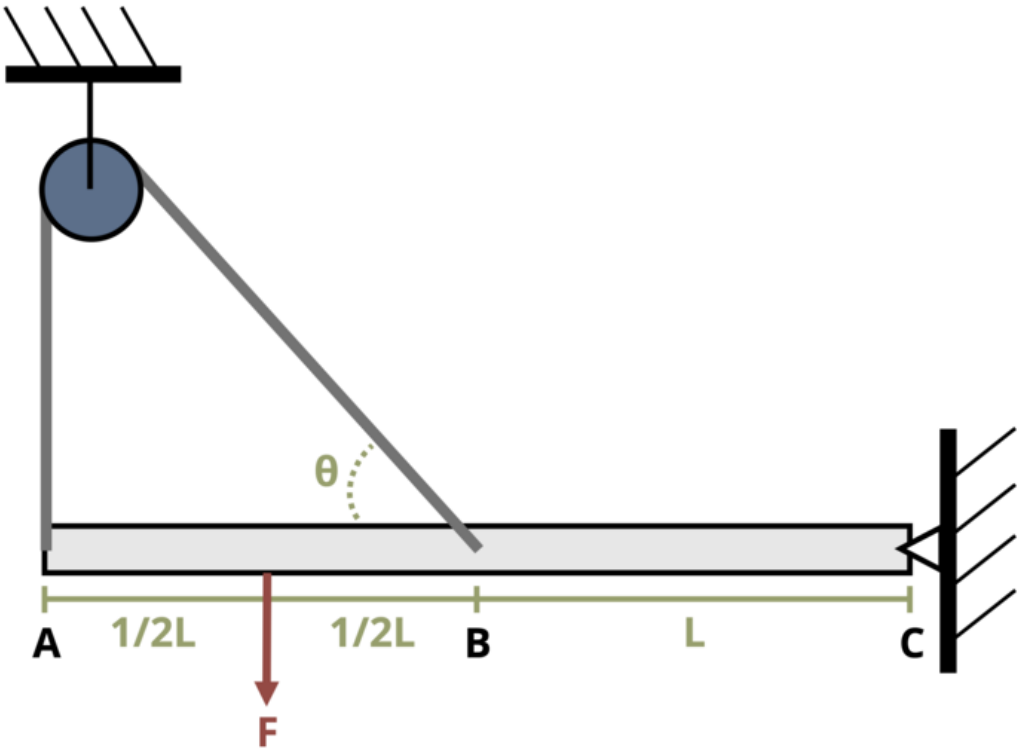
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="155"
```

```
F1=reactive.Value("__")
F2=reactive.Value("__")
F3=reactive.Value("__")
F4=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kPa", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A series of solid circular bars are connected together as shown

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(50, 200, 1)/10)
        F2.set(random.randrange(50, 200, 1)/10)
        F3.set(random.randrange(50, 140, 1)/10)
        F4.set(random.randrange(50, 200, 1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        F = F1()+F2()-F3()+F4()
```

```
        instr = F/227*10**4
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        # Store the most recent encoded attempt in a reactive value so it persists across su
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writ
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt
```

```
# App installation
app = App(app_ui, server)
```

Problem 2.13 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

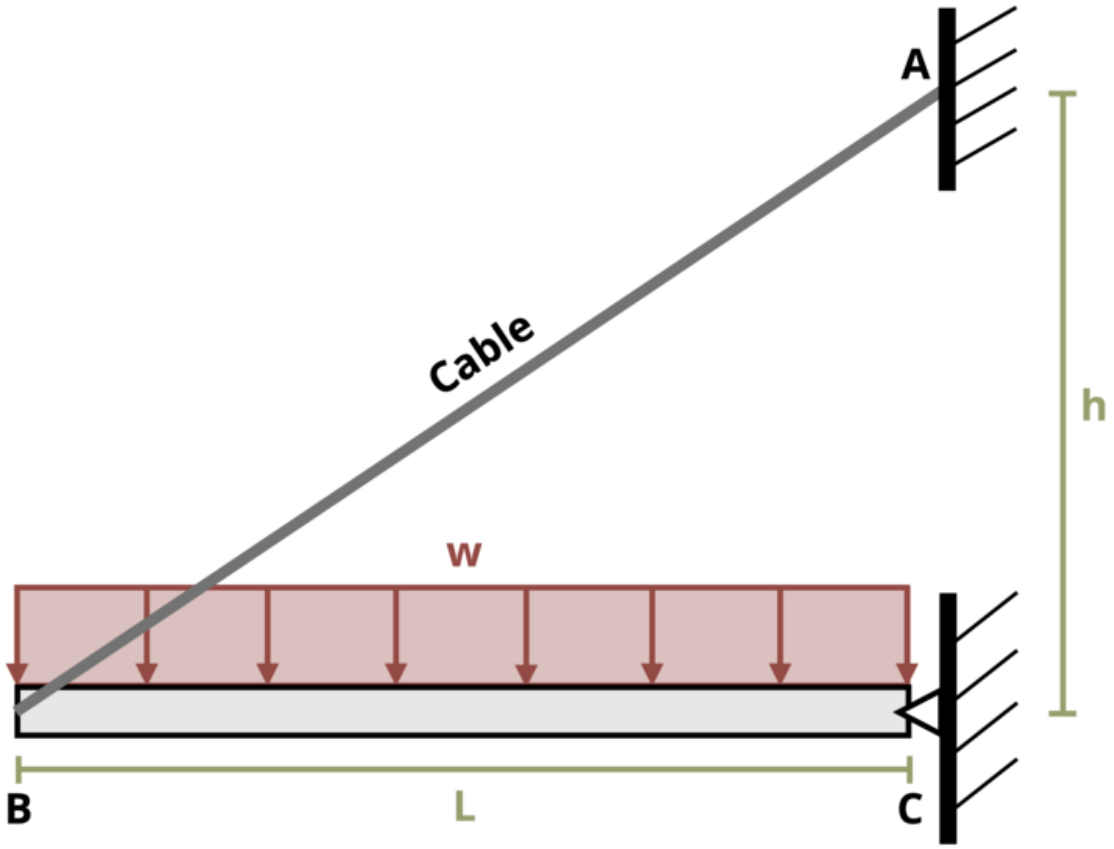
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
```

```
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="158"
F=reactive.Value("__")
theta=reactive.Value("__")
d=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your ansu
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A beam is held in place by a rope that loops over a pulley as sh

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(10, 100, 1))
        theta.set(random.randrange(33, 55, 1))
        d.set(random.randrange(15, 30, 1))
```

```
@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
    Fr = F()*1.5/(2+math.sin(theta()*math.pi/180))*1000
    r = d()/2000
    A = math.pi*r**2
    instr = Fr/A/10**6
    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
    random_middle = generate_random_letters(4)
    random_end = generate_random_letters(4)
    #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

    # Store the most recent encoded attempt in a reactive value so it persists across sub
    #session.encoded_attempt = reactive.Value(encoded_attempt)

    # Append the attempt data to the attempts list without the encoded attempt
    attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

    # Show feedback to the user.
    feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
    m = ui.modal(
        feedback,
        title="Feedback",
        easy_close=True
    )
    ui.modal_show(m)

#@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
async def download():
    # Start the CSV with the encoded attempt (without label)
    final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
    yield f"{final_encoded}\n\n"
```

```
        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.14 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

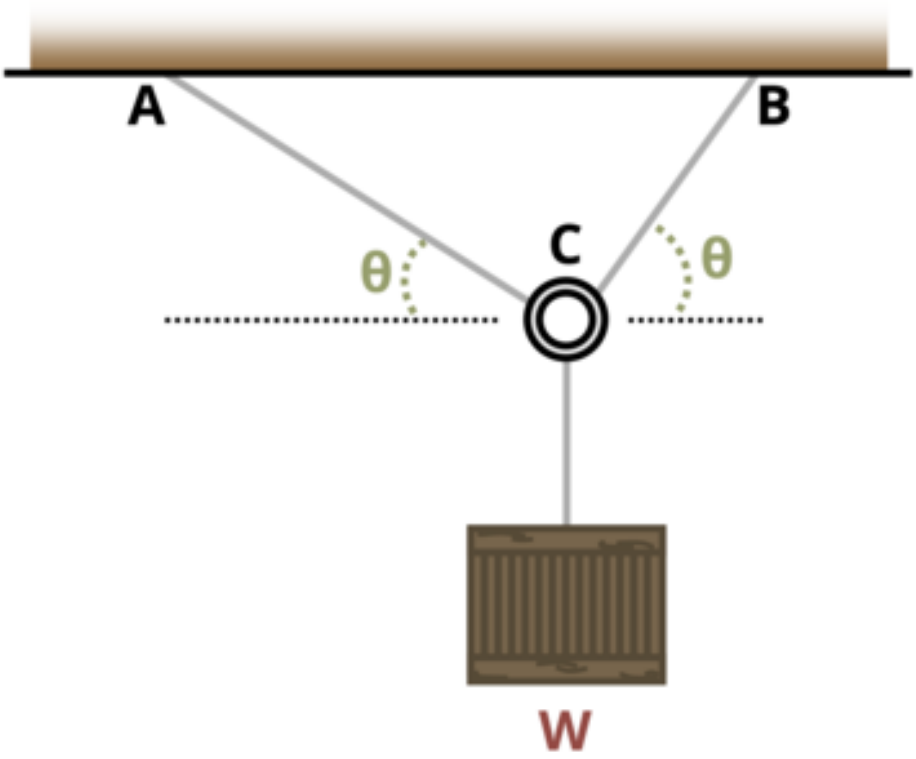
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
```

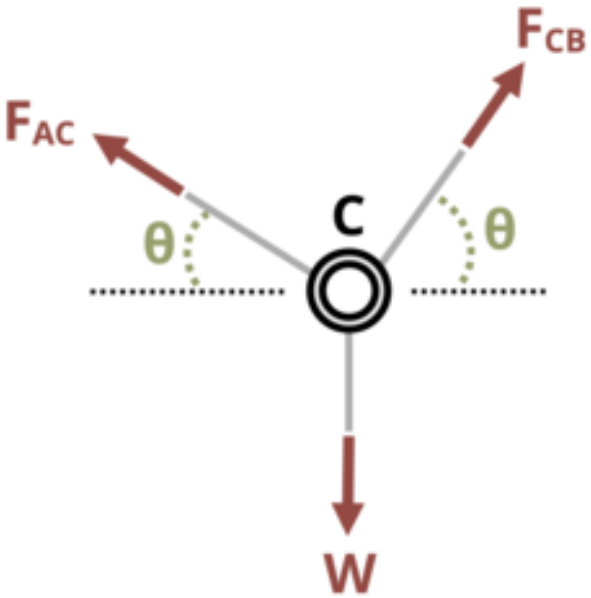
```
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="159"
w=reactive.Value("__")
d=reactive.Value("__")
L=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A cable supports a beam with a uniform distributed load w = {w()

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
```

```
        random.seed(101)
        w.set(random.randrange(10,50,1))
        d.set(random.randrange(15,30,1))
        L.set(random.randrange(10,30,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        theta = math.atan(1/L())
        Fc = (w()*L()*L()/2)/(L()*math.sin(theta))
        instr = Fc/(math.pi*(d()/2000)**2)/10**3

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
```

```
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.15 - Average Normal Stress



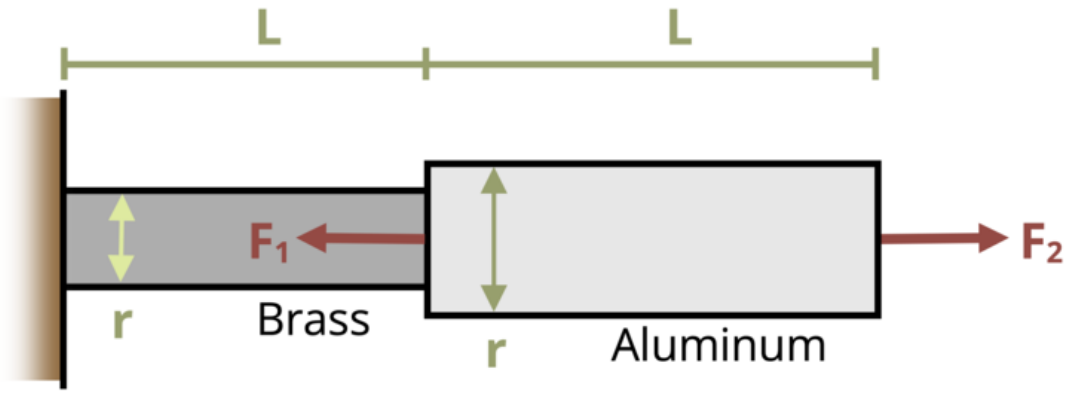


```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="160"
W=reactive.Value("__")
dAC=reactive.Value("__")
dBC=reactive.Value("__")
alpha=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of degrees", placeholder="Please enter your
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
```

```
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A box weighing W = {W()} lb is suspended by two cables as shown

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        W.set(random.randrange(300,1500,10))
        dBC.set(random.randrange(10,50,1)/10)
        dAC.set(dBC()+random.randrange(10,20,1)/10)
        alpha.set(random.randrange(50,60,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        AAC = (dAC()/2)**2*math.pi
        ACB = (dBC()/2)**2*math.pi
        ratio = ACB/AAC
        instr = math.acos(ratio*math.cos(alpha()*math.pi/180))*180/math.pi

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
```

```
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writ
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.16 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

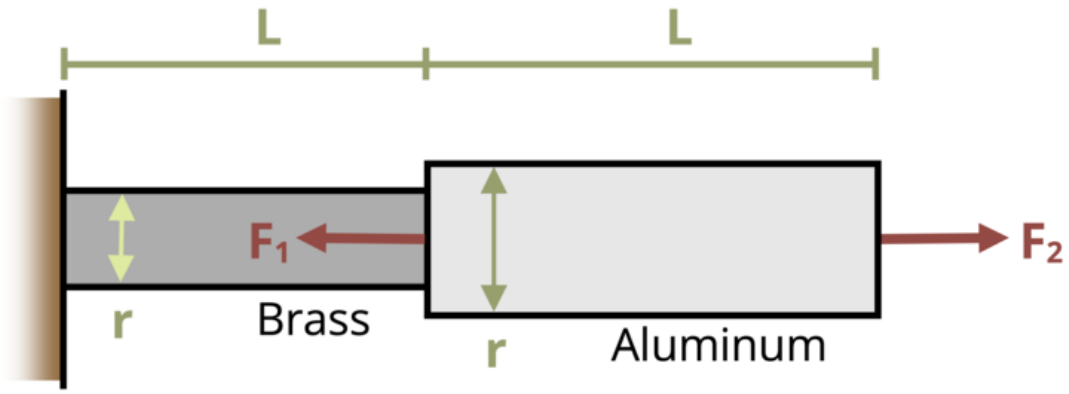
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "162"
F1 = reactive.Value("__")
```

```
F2 = reactive.Value("__")
d1 = reactive.Value("__")
d2 = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    ui.markdown(
        "**Please enter your ID number from your instructor and click to generate your proble
    ),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    ui.input_action_button(
        "generate_problem", "Generate Problem", class_="btn-primary"
    ),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of MPa", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"Two forces, F<sub>1</sub> = {F1()} KN and F<sub>2</sub> = {F2()} kN are ap
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F1.set(random.randrange(10, 50, 1))
```

```
        F2.set(F1() + random.randrange(10, 50, 1))
        d1.set(random.randrange(15, 30, 1))
        d2.set(d1() * 1.5)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.
        FB = F2() - F1()
        sigmaB = FB / ((d1() / 2) ** 2 * math.pi) * 10**3
        FA = F2()
        sigmaA = FA / ((d2() / 2) ** 2 * math.pi) * 10**3
        if sigmaB >= sigmaA:
            instr = sigmaB
        else:
            instr = sigmaA

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        # Store the most recent encoded attempt in a reactive value so it persists across su
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(
            f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
        )

        # Show feedback to the user.
        feedback = ui.markdown(
```

```
            f"Your answer of {input.answer()} is {check}."
        )
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = (
            session.encoded_attempt()
            if session.encoded_attempt is not None
            else "No attempts"
        )
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(
                0.25
            )  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.17 - Average Normal Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

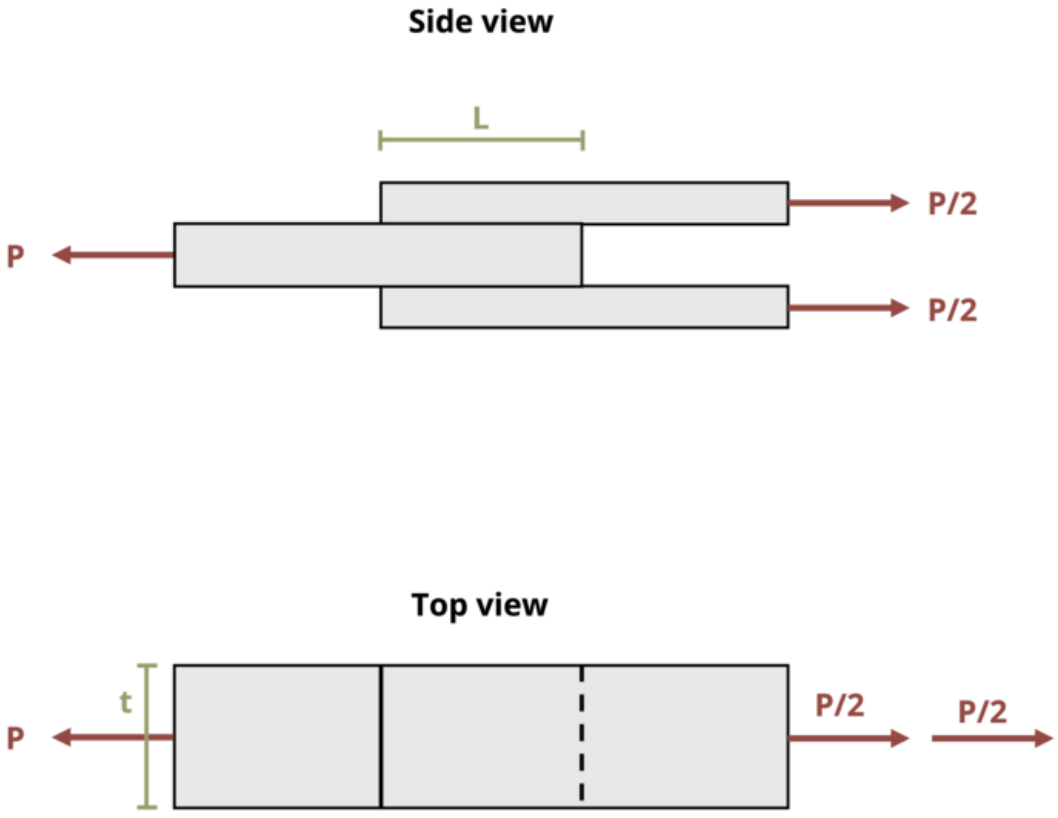
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "163"
w0 = reactive.Value("__")
```

```
L = reactive.Value("__")
AA = reactive.Value("__")
AB = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    ui.markdown(
        "**Please enter your ID number from your instructor and click to generate your proble
    ),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    ui.input_action_button(
        "generate_problem", "Generate Problem", class_="btn-primary"
    ),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of ksi", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A beam structure supported by a tube at each end is loaded with a distribut
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        w0.set(random.randrange(20,100,1)/10)
```

```
        L.set(random.randrange(6,15,3))
        AA.set(random.randrange(20,40,1)/10)
        AB.set(random.randrange(50,70,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.
        FB = w0()*L()*0.5*2/3
        FA = w0()*L()*0.5-FB
        sigmaA = FA/AA()
        sigmaB = FB/AB()
        if sigmaA >= sigmaB:
            instr = sigmaA
        else:
            instr = sigmaB

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        # Store the most recent encoded attempt in a reactive value so it persists across su
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(
            f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
        )

        # Show feedback to the user.
        feedback = ui.markdown(
```

```
            f"Your answer of {input.answer()} is {check}."
        )
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = (
            session.encoded_attempt()
            if session.encoded_attempt is not None
            else "No attempts"
        )
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(
                0.25
            )  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.21 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

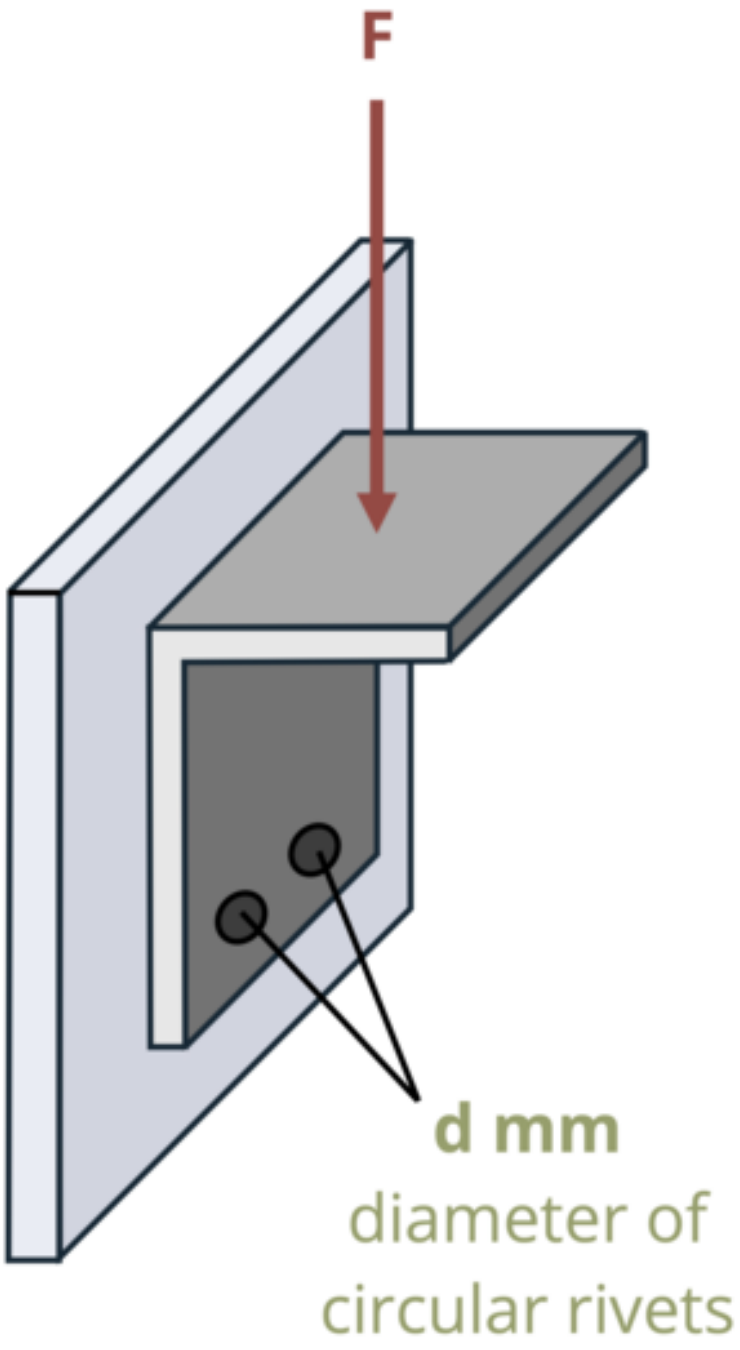
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
```

```
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="164"
 fail=reactive.Value("__")
L=reactive.Value("__")
t=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kips", placeholder="Please enter your ans
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)


def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A double lap joint is glued together using glue with a shear str

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
```

```
        random.seed(101)
         fail.set(random.randrange(7000, 9000, 100))
        L.set(random.randrange(40, 100, 1)/10)
        t.set(random.randrange(40, 100, 1)/10)


    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s

        A = L()*t()*2
        instr= ( fail()*A)/1000
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
```

```
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt


# App installation
app = App(app_ui, server)
```

Problem 2.22 - Average Shear Stress



```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="165"
F=reactive.Value("__")
d=reactive.Value("__")


attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)


def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
```

```
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A bracket is attached to a wall with two circular rivets of dia

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(30, 100, 1))
        d.set(random.randrange(10, 40, 1))


    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s

        A = math.pi*(d()/(1000*2))**2
        instr= ((F()/2)/A)/1000
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
```

```
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writ
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt


# App installation
app = App(app_ui, server)
```

Problem 2.23 - Average Shear Stress

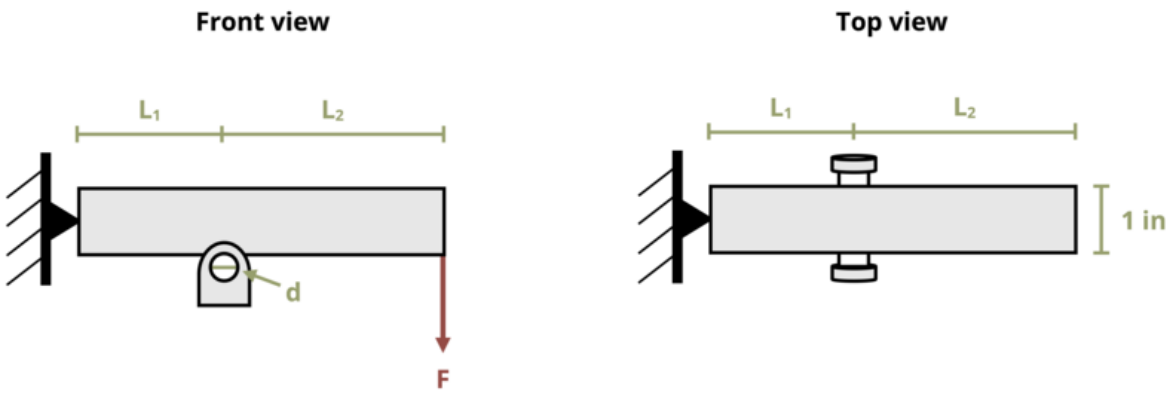


Figure 2: Figure 1: A square bar of length is pinned at one end and rests on a circular rod.

[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

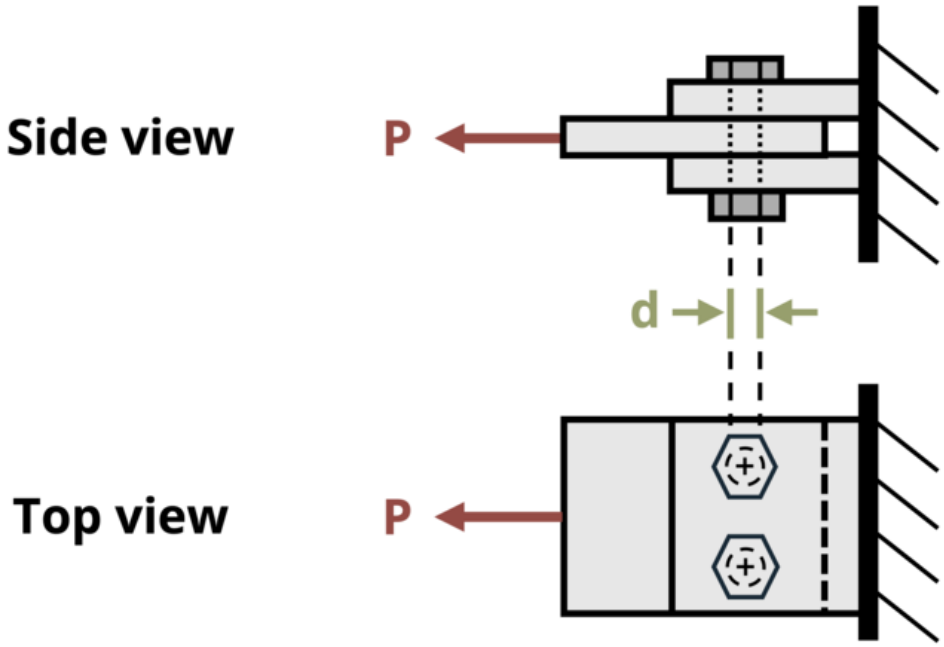
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```
problem_ID="168"
L1=reactive.Value("__")
L2=reactive.Value("__")
d=reactive.Value("__")
F=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A square bar of length L<sub>1</sub> = {L1()} in. and L<sub>2</s

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L1.set(random.randrange(50, 150, 1)/10)
        L2.set(round(L1() * 1.4, 1))
        d.set(random.randrange(4, 9, 1)/10)
        F.set(random.randrange(30, 100, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
```

```
        M = F()*(L1()+L2())
        R = M/L1()
        A = math.pi*(d()/2)**2
        instr= R/(2*A)
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
```

```
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.24 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

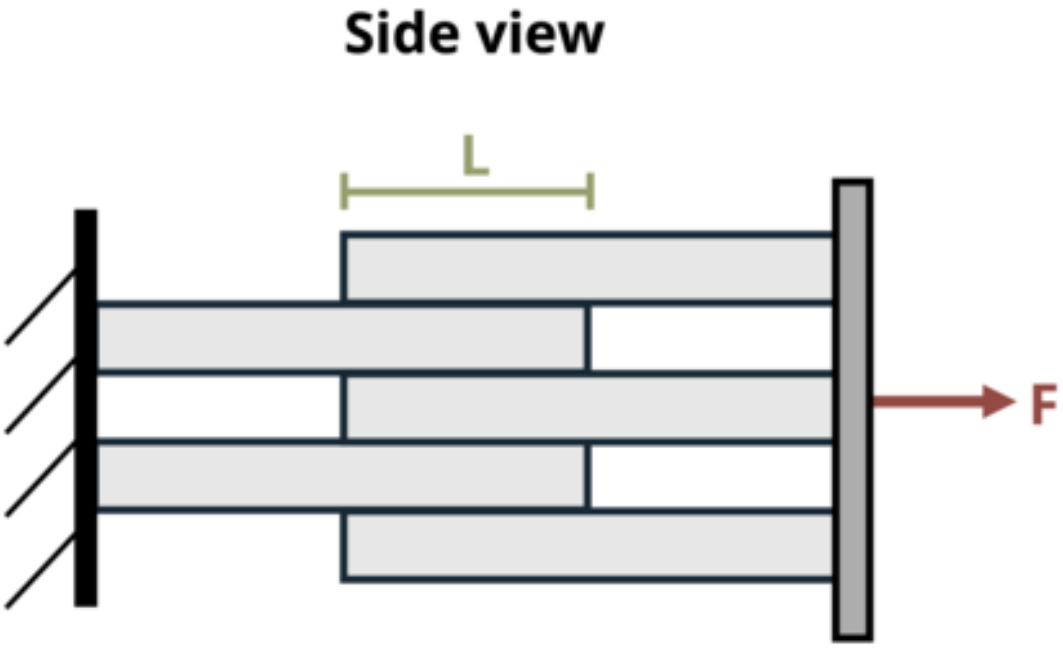
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

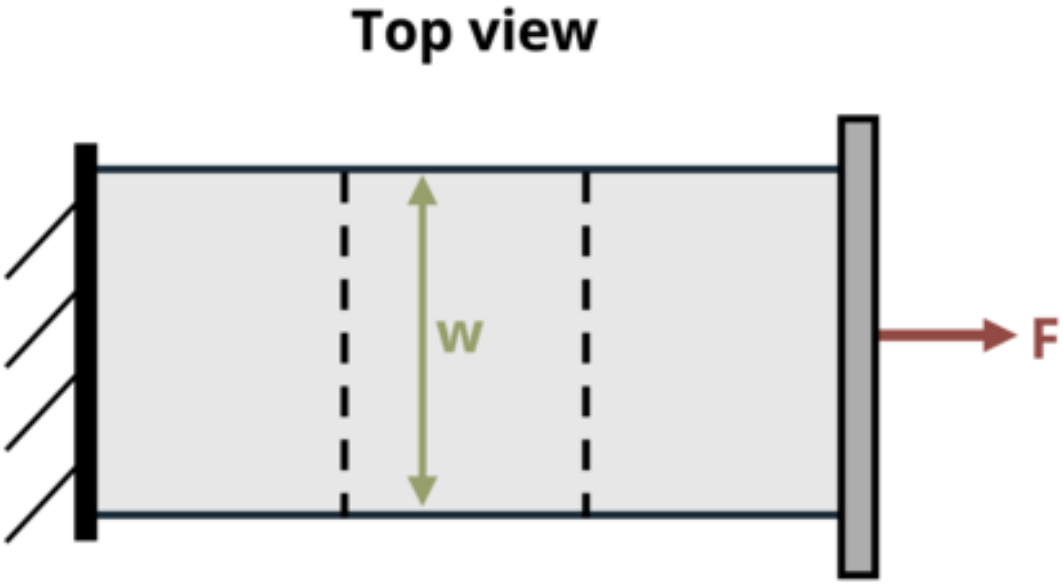
```
def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "170"
d = reactive.Value("__")
tau_fail = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    ui.markdown(
        "**Please enter your ID number from your instructor and click to generate your proble
    ),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    ui.input_action_button(
        "generate_problem", "Generate Problem", class_="btn-primary"
    ),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of kN", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A double lap joint is held together with two steel bolts. Each bolt has a c
            )
        ]

    #@reactive.Effect
```

```
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(10,30,1))
        tau_fail.set(random.randrange(200,300,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.
        instr = tau_fail()*2*2*((d()/2)**2*math.pi)/10**3

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(
            f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
        )

        # Show feedback to the user.
        feedback = ui.markdown(
            f"Your answer of {input.answer()} is {check}."
        )
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)
```

```
    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = (
            session.encoded_attempt()
            if session.encoded_attempt is not None
            else "No attempts"
        )
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(
                0.25
            )  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.25 - Average Shear Stress



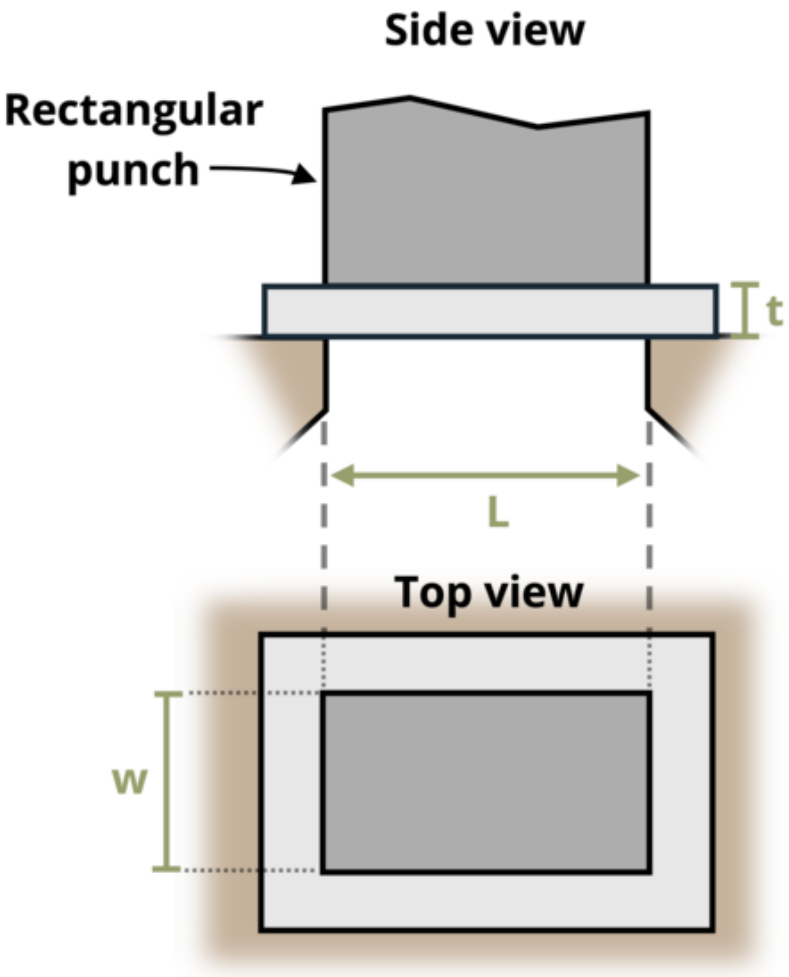


```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path


def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "173"
L = reactive.Value("__")
w = reactive.Value("__")
F = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    ui.markdown(
        "**Please enter your ID number from your instructor and click to generate your proble
    ),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    ui.input_action_button(
        "generate_problem", "Generate Problem", class_="btn-primary"
    ),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of MPa", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
```

```
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A lap joint is constructed with layers as shown. If dimensions L = {L()} mm
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        L.set(random.randrange(10, 50, 1))
        w.set(round(L() * 1.6,1))
        F.set(random.randrange(5, 75, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        A = w()/1000*L()/1000*4
        instr = F()*1000/A/10**6

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
```

```
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.26 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

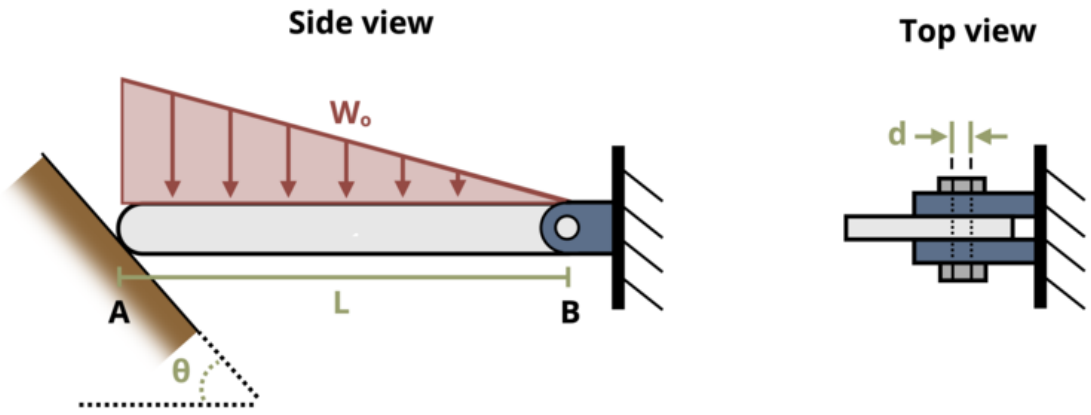
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]
```

```
from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path


def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "174"
w = reactive.Value("__")
L = reactive.Value("__")
t = reactive.Value("__")
tau_fail = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    ui.markdown(
        "**Please enter your ID number from your instructor and click to generate your proble
    ),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    ui.input_action_button(
        "generate_problem", "Generate Problem", class_="btn-primary"
    ),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of kip", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)
```

```
    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A rectangular hole punch is configured to punch a hole of L = {L()} in. by
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        w.set(random.randrange(20,60,1)/10)
        L.set(round(w()*2,1))
        t.set(random.randrange(10,75,1)/100)
        tau_fail.set(random.randrange(5,40,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        A = (2*L()+2*w())*t()
        instr = tau_fail()*A

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c
```

```
        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.27 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

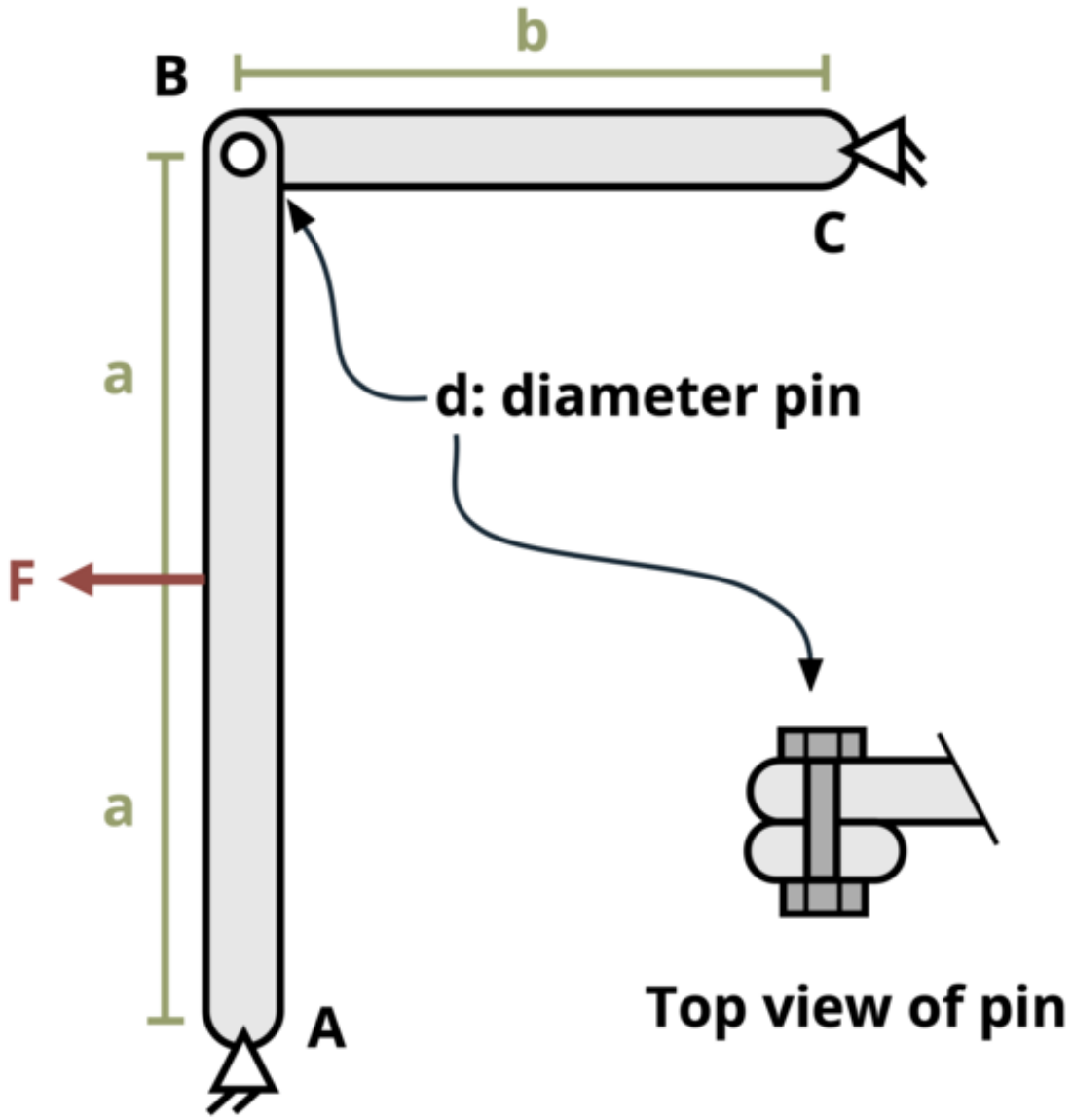
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "175"
theta = reactive.Value("__")
d = reactive.Value("__")
```

```
w0 = reactive.Value("__")
L = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    ui.markdown(
        "**Please enter your ID number from your instructor and click to generate your proble
    ),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    ui.input_action_button(
        "generate_problem", "Generate Problem", class_="btn-primary"
    ),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of MPa", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"One side of a beam rests on a sloping wall at angle θ = {theta()}°. The oth
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        theta.set(random.randrange(55,65,1))
        d.set(random.randrange(10,30,1))
        w0.set(random.randrange(5,20,1))
```

```
        L.set(random.randrange(10,50,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        By = w0()*L()/2/3
        Ay = w0()*L()/2-By
        Ax = Ay*math.tan(theta()*math.pi/180)
        Bx = Ax
        Babs = (By**2+Bx**2)**0.5
        P = Babs/2
        instr = P/((d()/2)**2*math.pi)*10**3

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        # Store the most recent encoded attempt in a reactive value so it persists across su
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
```

```
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writ
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.28 - Average Shear Stress



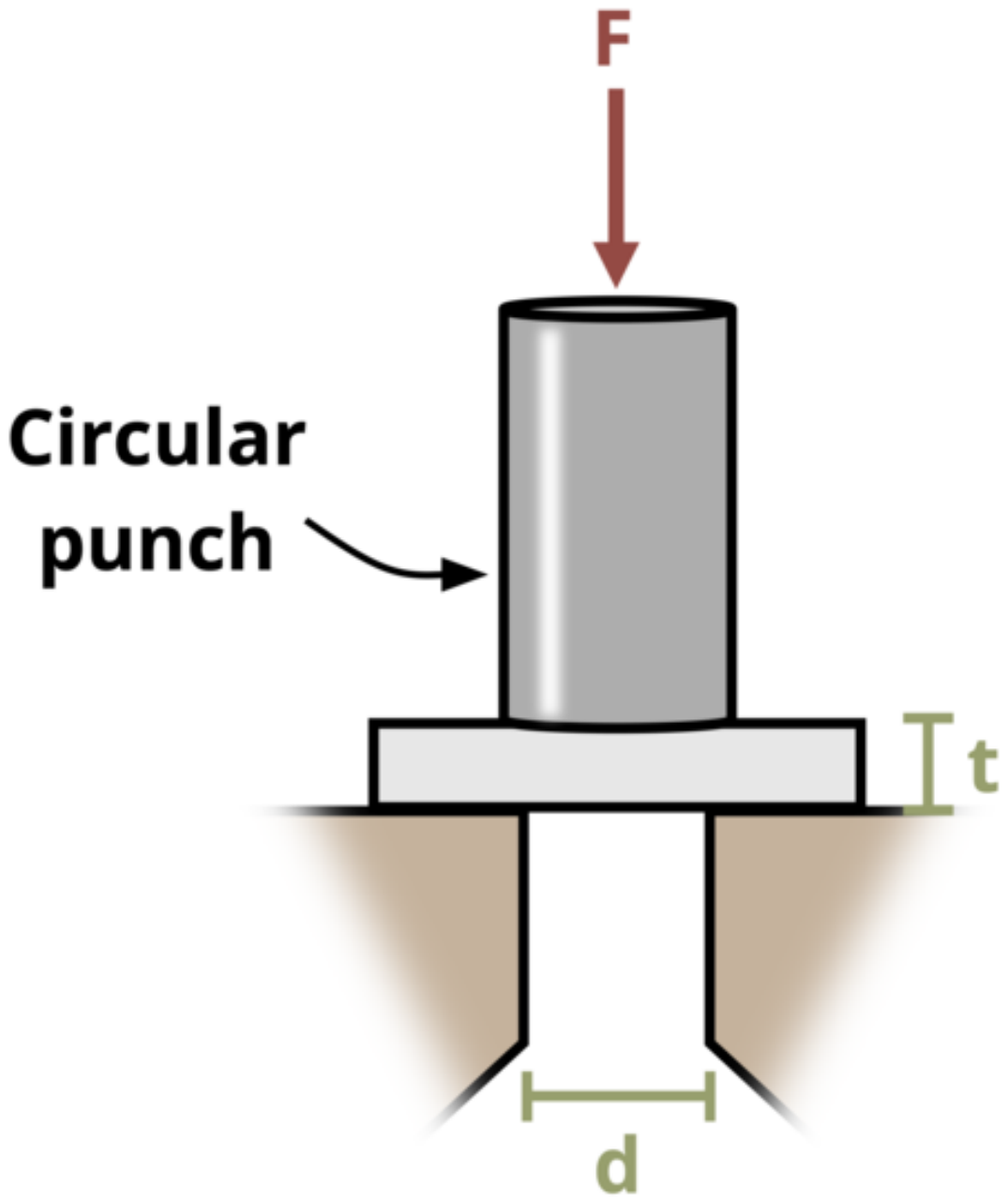

```
#| standalone: true
#| viewerHeight: 600
```

```
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
problem_ID="177"
d=reactive.Value("__")
a=reactive.Value("__")
b=reactive.Value("__")
F=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two structural members are connected with a pin of diameter d =
```

```
    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(20,100,5)/100)
        a.set(random.randrange(10,50,1)/10)
        b.set(round(a(),1)*round(random.randrange(12,17,1)/10))
        F.set(random.randrange(10,100,1)/10)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        FB = F()/2
        instr = FB/((d()/2)**2*math.pi)

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
```

```
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writ
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.29 - Average Shear Stress



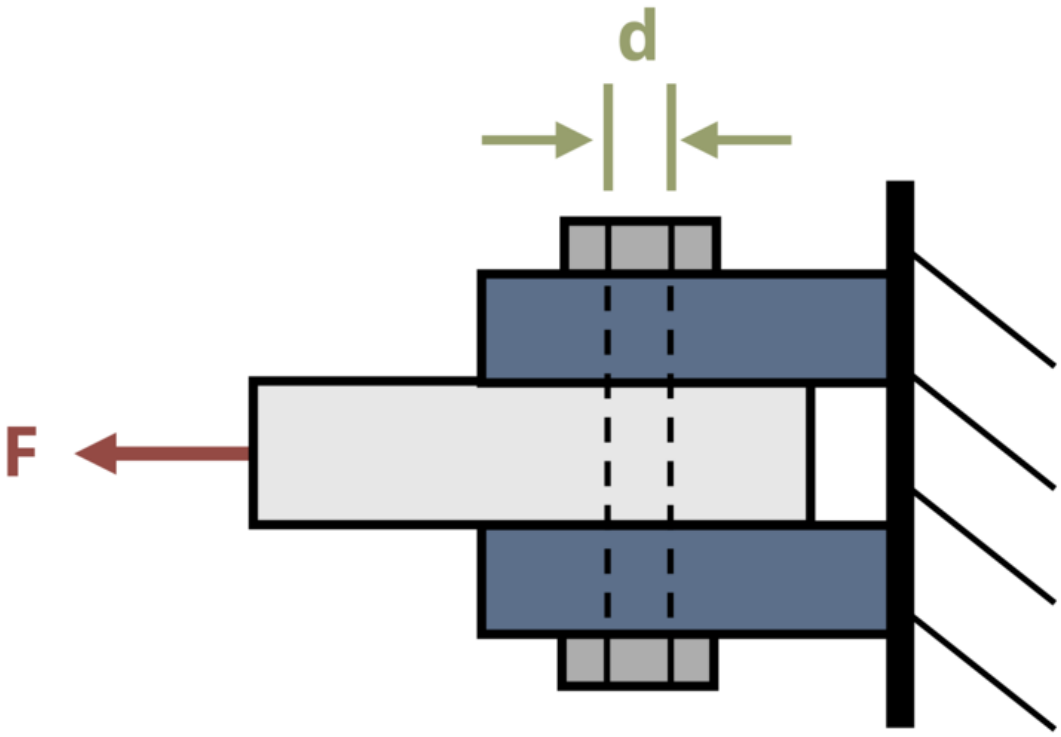
[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
problem_ID="178"
d=reactive.Value("__")
t=reactive.Value("__")
tau_fail=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kips", placeholder="Please enter your an
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
```

```
        return[ui.markdown(f"A circular hole punch is configured to punch a hold of diameter

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(25,95,5)/100)
        t.set(random.randrange(3,15,1)/10)
        tau_fail.set(random.randrange(20,40,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        A = t()*d()*math.pi
        instr = tau_fail()*A

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
```

```
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.30 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

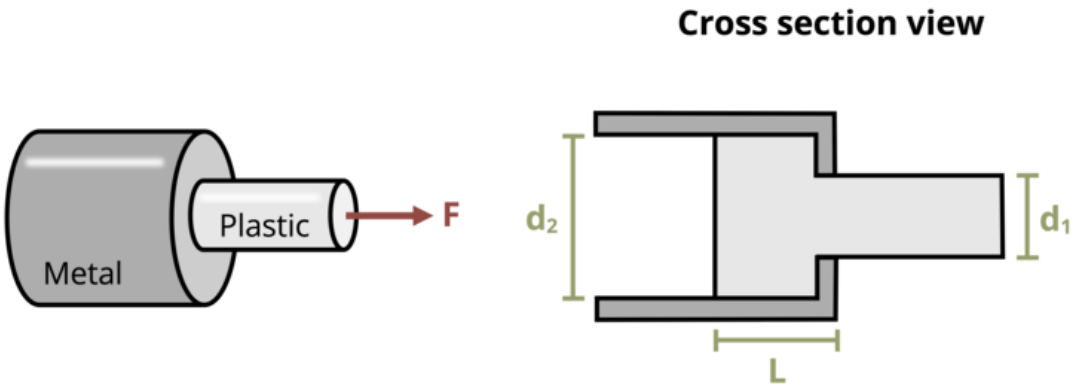
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
```

```
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="179"
d=reactive.Value("__")
F=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of MPa", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A bolt of diameter d = {d()} mm is used at a pin support in a st

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(20,50,1))
        F.set(random.randrange(50,150,1))
```

```
    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        V = F()/2
        A = math.pi*(d()/2)**2
        instr = V/(A)*10**3

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
```

```
        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.31 - Average Shear Stress



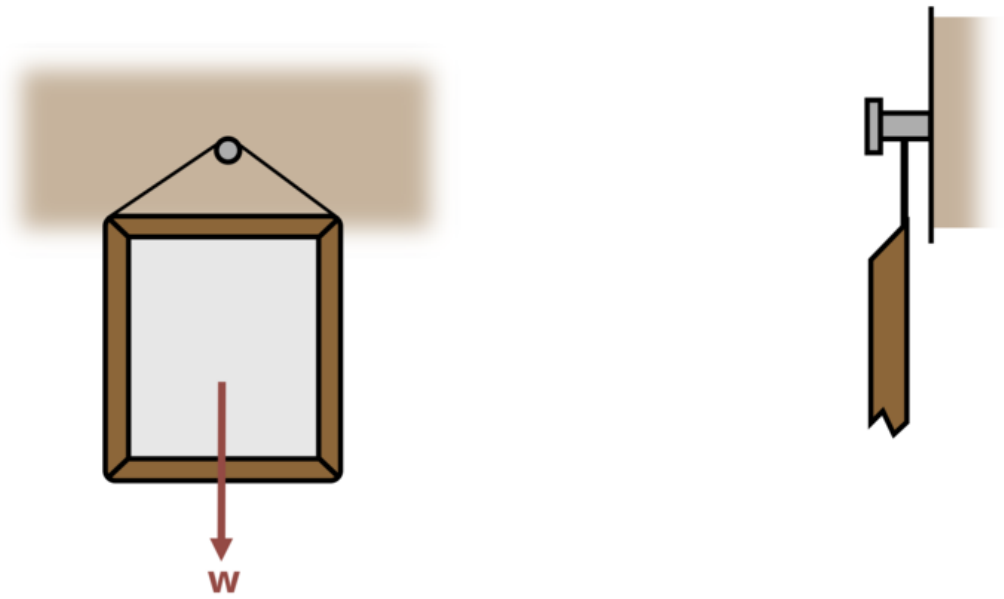
[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
problem_ID="181"
d1=reactive.Value("__")
d2=reactive.Value("__")
L=reactive.Value("__")
F=reactive.Value("__")
```

```
attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A plastic cylindrical peg is constrained by a metal cap as shown

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d1.set(random.randrange(2,10,1)/10)
        d2.set(round(d1(),2)*round(random.randrange(12,18,1)/10))
        L.set(random.randrange(5,15,1)/10)
        F.set(random.randrange(10,50,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        A = d1()*math.pi*L()
        instr = F()/A

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
```

```
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.32 - Average Shear Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

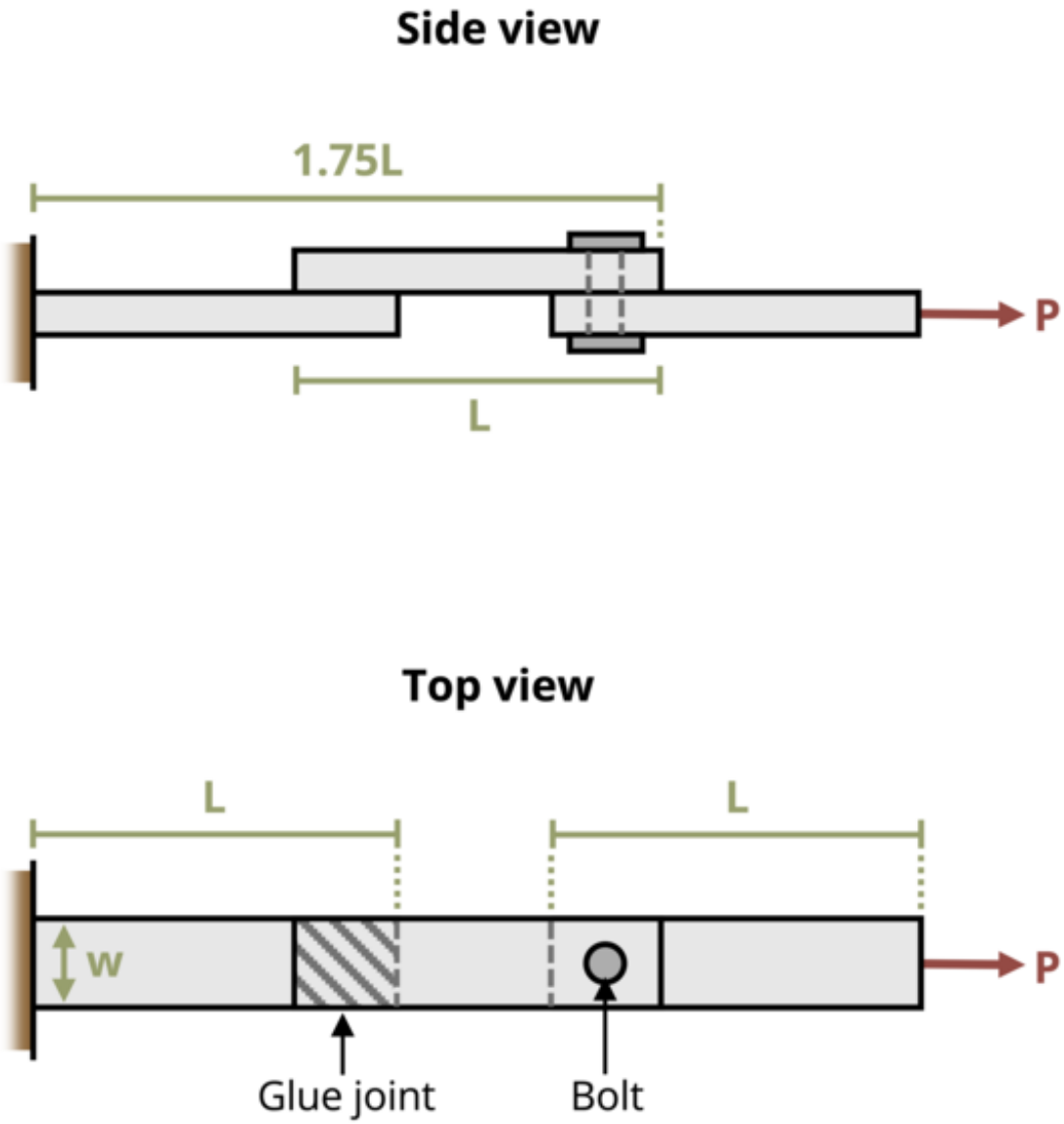
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
```

```
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="182"
W=reactive.Value("__")
tau_fail=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of in", placeholder="Please enter your answe
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A plaque weighing W = {W()} lb is hung from a rafter by a single

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        W.set(random.randrange(50,150,1))
        tau_fail.set(random.randrange(20,40,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        instr = (4*W()/(math.pi*tau_fail()*1000))**0.5
```

```
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt
```

```
# App installation
app = App(app_ui, server)
```

Problem 2.33 - Average Shear Stress



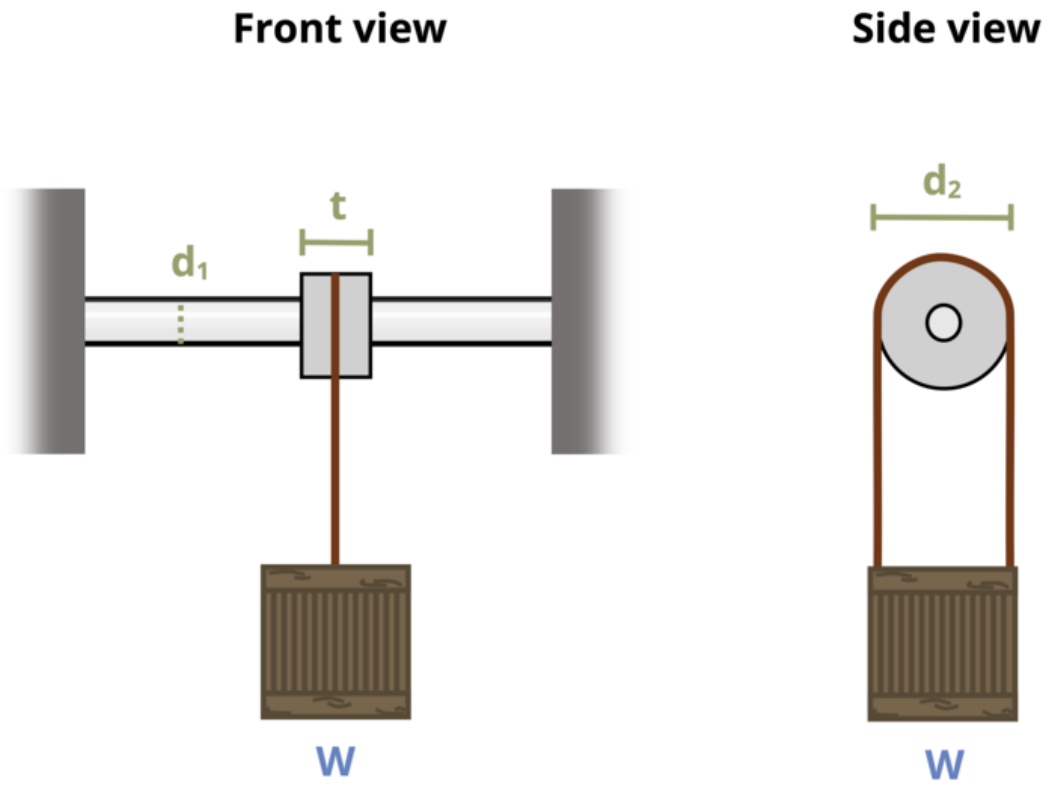

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]
```

```
from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "167"
L = reactive.Value("__")
w = reactive.Value("__")
tau_glue = reactive.Value("__")
d = reactive.Value("__")
tau_bolt = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    ui.markdown(
        "**Please enter your ID number from your instructor and click to generate your proble
    ),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    ui.input_action_button(
        "generate_problem", "Generate Problem", class_="btn-primary"
    ),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of lb", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)
```

```
@output
@render.ui
def ui_problem_statement():
    randomize_vars()
    return [
        ui.markdown(
            f"Three bars of length L = {L()} in. and width w = {w()} in. are attached to
        )
    ]

#@reactive.Effect
#@reactive.event(input.generate_problem)
def randomize_vars():
    random.seed(101)
    L.set(random.randrange(4,10,1))
    w.set(random.randrange(10,30,1)/10)
    tau_glue.set(random.randrange(500,1000,10))
    d.set(random.randrange(3,8,1)/10)
    tau_bolt.set(random.randrange(50,150,1)/10)

@reactive.Effect
@reactive.event(input.submit)
def _():
    attempt_counter.set(
        attempt_counter() + 1
    )  # Increment the attempt counter on each submission.
    Pg = tau_glue()*w()*.25*L()
    Pb = tau_bolt()*1000*((d()/2)**2*math.pi)
    if Pg >= Pb:
        instr = Pb
    else:
        instr = Pg

    if math.isclose(float(input.answer()), instr, rel_tol=0.01):
        check = "*Correct*"
        correct_indicator = "JL"
    else:
        check = "*Not Correct.*"
        correct_indicator = "JG"

    # Generate random parts for the encoded attempt.
    random_start = generate_random_letters(4)
```

```
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(
            f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
        )

        # Show feedback to the user.
        feedback = ui.markdown(
            f"Your answer of {input.answer()} is {check}."
        )
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = (
            session.encoded_attempt()
            if session.encoded_attempt is not None
            else "No attempts"
        )
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(
                0.25
            )  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.38 - Bearing Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]
from shiny import App, render, ui, reactive
import random
import asyncio
import io
```

```
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="166"
W=reactive.Value("__")
d1=reactive.Value("__")
d2=reactive.Value("__")
t=reactive.Value("__")
attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)


def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A crate of weight {W()} = lb hangs from a solid circular metal r

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
```

```
        W.set(random.randrange(4000, 9000, 100))
        d1.set(random.randrange(5, 30, 1)/10)
        d2.set(d1()*3)
        t.set(d1()*2)
    @reactive.Effect
    @reactive.event(input.submit)
    def _():

        instr= W()/(d1()*t()*1000)

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
```

```
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt


# App installation
app = App(app_ui, server)
```

Problem 2.39 - Bearing Stress

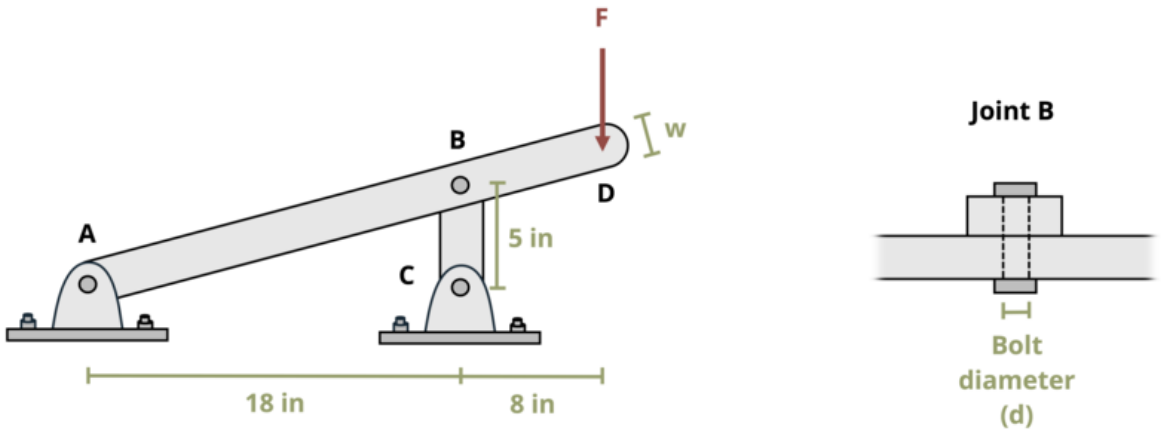


Figure 3: Figure 1: A link mechanism is connected with pins.

[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

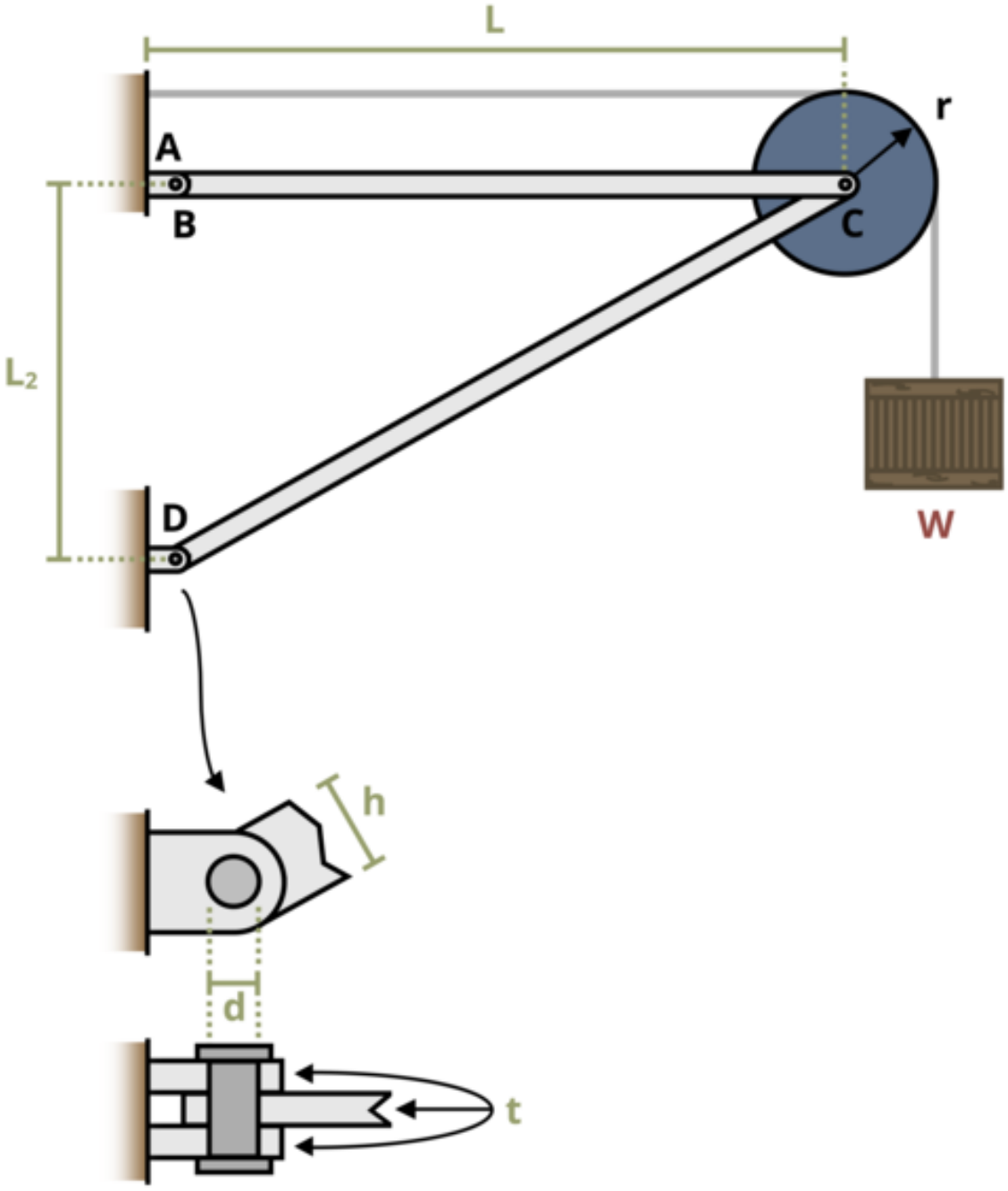
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```
problem_ID="180"
F=reactive.Value("__")
d=reactive.Value("__")
t=reactive.Value("__")
w=reactive.Value("__")



attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)


def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A link mechanism is connected with pins of diameter d = {d()} in

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(200, 900, 10))
        d.set(random.randrange(20, 150, 10)/100)
        t.set(random.randrange(2, 10, 1)/10)
        w.set(round(d()*2, 2))
```

```
    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        Fb=((18+8)*F())/18
        instr= Fb/(d()*t())/1000
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"
```

```
        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt


# App installation
app = App(app_ui, server)
```

Problem 2.40 - Bearing Stress



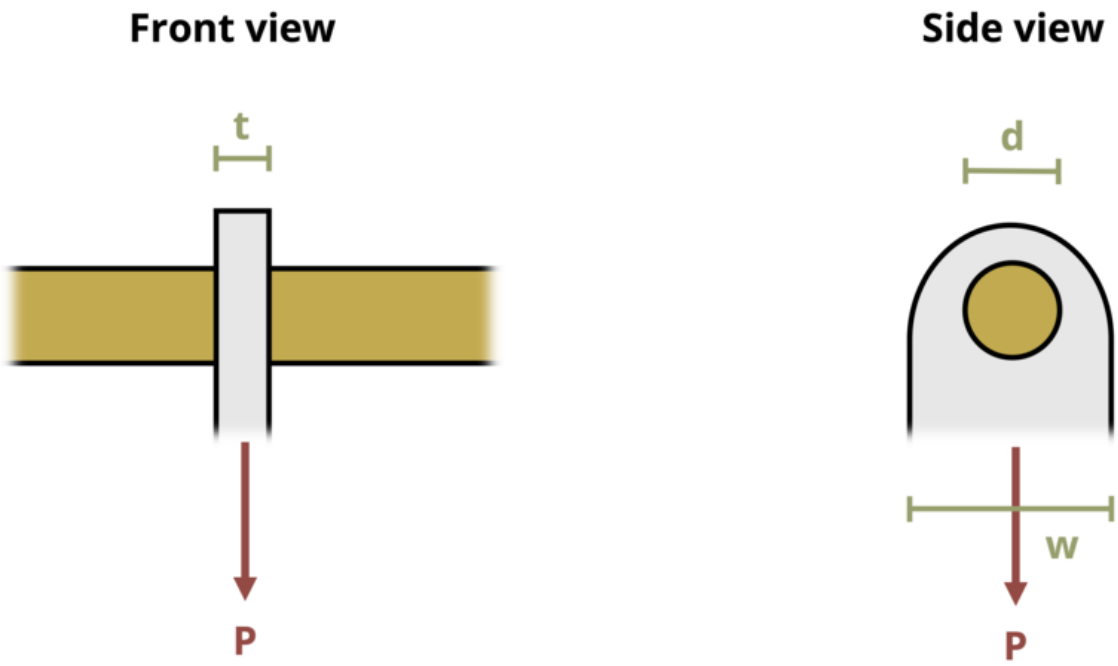
[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="161"
W=reactive.Value("__")
t=reactive.Value("__")
d=reactive.Value("__")
h=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of ksi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
```

```
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A crate weighing W = {W()} kips is supported by the structure as

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        W.set(random.randrange(5,100,1))
        t.set(random.randrange(25, 75, 1)/100)
        d.set(t()*4)
        h.set(d()*2)

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        theta = 29.05*math.pi/180
        FDC = W()/math.sin(theta)
        instr= FDC/(d()*t())
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
```

```
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.41 - Bearing Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

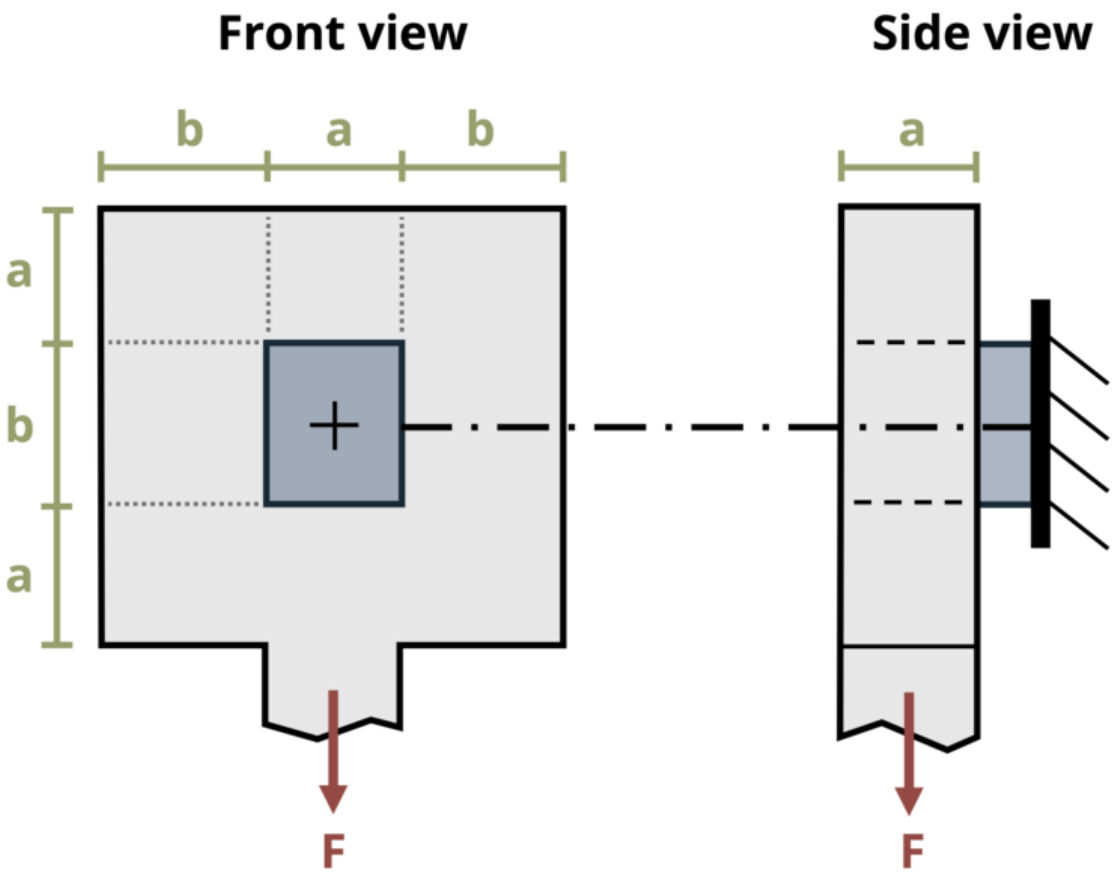
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path
```

```
def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="169"
d=reactive.Value("__")
t=reactive.Value("__")
w=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate y
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of kips", placeholder="Please enter your an
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A steel connector plate is hung from a brass rod of diameter d =

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        d.set(random.randrange(8, 20, 1)/10)
        t.set(random.randrange(3, 7, 1)/10)
        w.set((d()*2))

    @reactive.Effect
    @reactive.event(input.submit)
```

```
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s
        instr= 70*t()*d()
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
```

```
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.42 - Bearing Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

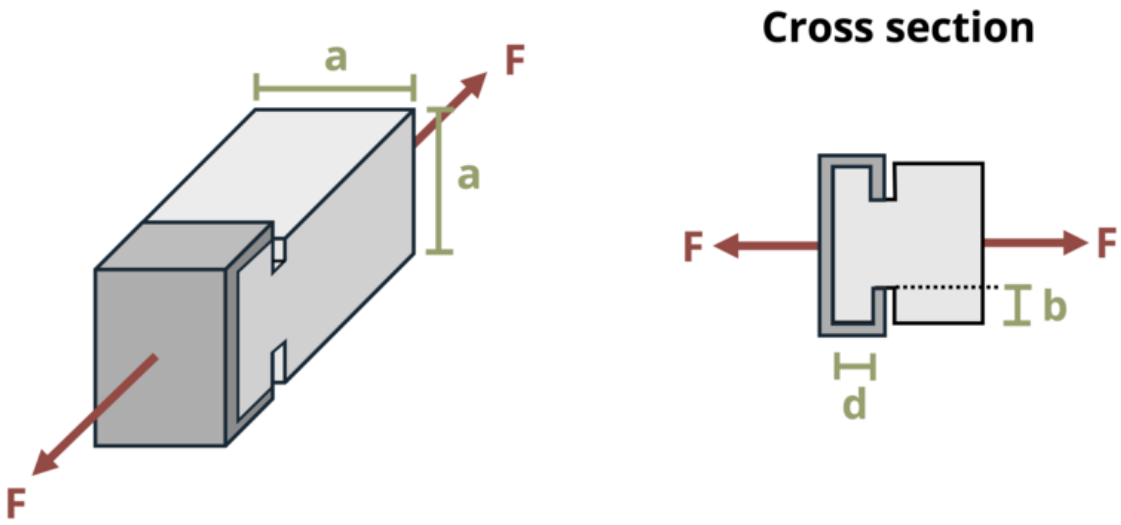
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
```

```
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID = "172"
a = reactive.Value("__")
b = reactive.Value("__")
F = reactive.Value("__")

attempts = ["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    ui.markdown(
        "**Please enter your ID number from your instructor and click to generate your proble
    ),
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    ui.input_action_button(
        "generate_problem", "Generate Problem", class_="btn-primary"
    ),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text(
        "answer", "Your Answer in units of MPa", placeholder="Please enter your answer"
    ),
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
```

```
        return [
            ui.markdown(
                f"A hanger is mounted to a wall with a rectangular bar as shown. Let dimensio
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        a.set(random.randrange(100,300,2)/10)
        b.set(round(a()*1.2,1))
        F.set(random.randrange(10,70,1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.
        sigmaN = F()/2/(b()*a())*10**3
        sigmaBH = F()/(a()*a())*10**3
        tauH = F()/2/(a()*a())*10**3
        tauR = F()/(a()*b())*10**3
        instr = max(sigmaN, sigmaBH, tauH, tauR)

        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)
```

```
        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(
            f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"
        )

        # Show feedback to the user.
        feedback = ui.markdown(
            f"Your answer of {input.answer()} is {check}."
        )
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = (
            session.encoded_attempt()
            if session.encoded_attempt is not None
            else "No attempts"
        )
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writ
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(
                0.25
            )  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.43 - Bearing Stress



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

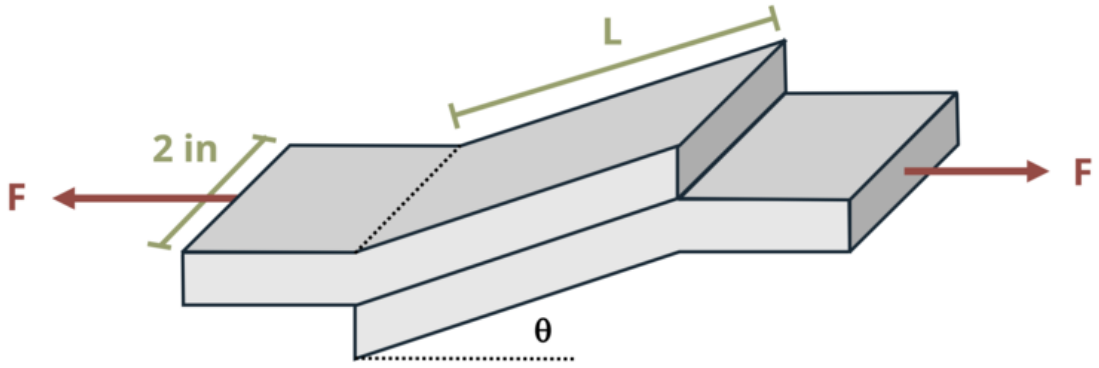
```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return "".join(random.choice(string.ascii_lowercase) for _ in range(length))
```

```
problem_ID = "176"
sigma_s = reactive.Value("__")
sigma_b = reactive.Value("__")
F = reactive.Value("__")
a = reactive.Value("__")
attempts = ["Timestamp,Attempt,Answer1,Answer2,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID", "", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer1", "Your Answer 1 in units of in", placeholder="Please enter your a
    ui.input_text("answer2", "Your Answer 2 in units of in", placeholder="Please enter your a
    ui.input_action_button("submit", "Check Answers", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return [
            ui.markdown(
                f"A metal end cap is secured to a wood structural member with grooves as shou
            )
        ]

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        sigma_s.set(random.randrange(20,50,1)/10)
        sigma_b.set(sigma_s()*2)
        F.set(random.randrange(15,40,1))
        a.set(random.randrange(50,100,1)/10)
```

```
    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(
            attempt_counter() + 1
        )  # Increment the attempt counter on each submission.

        # Calculate the instructor's answer and determine if the user's answer is correct.
        P = F()/2
        Ab = P/sigma_b()
        instr1 = Ab/a()
        As = P/sigma_s()
        instr2 = As/a()

        correct1 = math.isclose(float(input.answer1()), instr1, rel_tol=0.01)
        correct2 = math.isclose(float(input.answer2()), instr2, rel_tol=0.01)

        if correct1 and correct2:
            check = f"{'both correct.'}"
        elif correct1:
            check = f" {'correct for answer 1 and incorrect for answer 2.'}"
        elif correct2:
            check = f"{'incorrect for answer 1 and correct for answer 2.'}"
        else:
            check = f"{'both incorrect.'}"

        correct_indicator = "JL" if correct1 and correct2 else "JG"

        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{c

        #session.encoded_attempt = reactive.Value(encoded_attempt)
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer1()}, {input.a

        feedback = ui.markdown(f"Your answers of {input.answer1()} and {input.answer2()} are
        m = ui.modal(feedback, title="Feedback", easy_close=True)
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
```

```
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"
        yield "Timestamp,Attempt,Answer1,Answer2,Feedback\n"
        for attempt in attempts[1:]:
            await asyncio.sleep(0.25)
            yield attempt

app = App(app_ui, server)
```

Problem 2.47 - Stress on an Inclined Plane



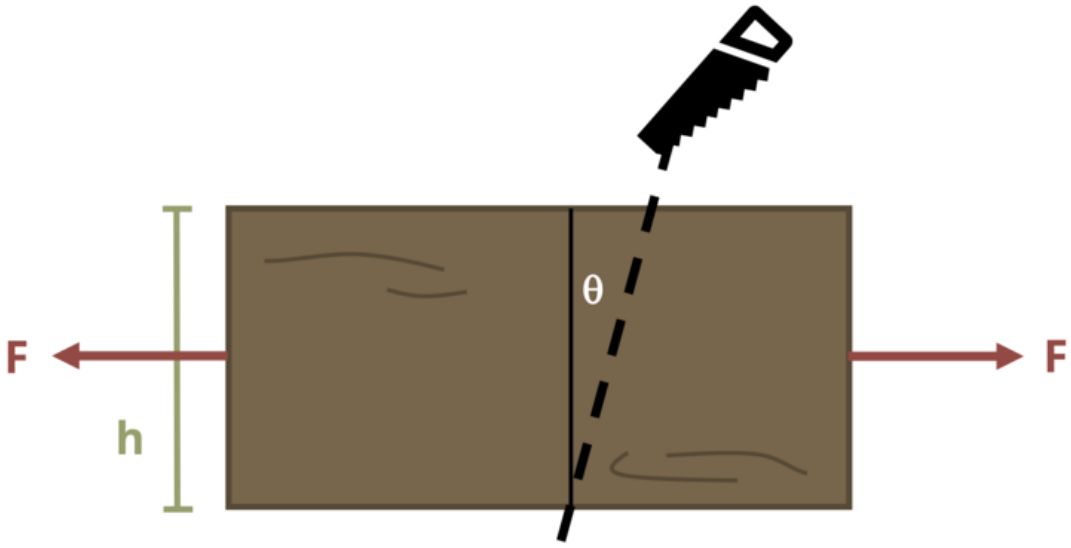
[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="153"
F=reactive.Value("__")
L=reactive.Value("__")
```

```
Θ=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)

def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"Two slanted brackets are glued together as shown. If F = {F()} l

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(200, 800, 10))
        L.set(random.randrange(20, 80, 1)/10)
        Θ.set(random.randrange(15, 30, 1))

    @reactive.Effect
    @reactive.event(input.submit)
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s

        instr= (F()*math.cos(math.radians(Θ()))/(L()*2))
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
```

```
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt

# App installation
app = App(app_ui, server)
```

Problem 2.48 - Stress on an Inclined Plane



[Problem adapted from © Kurt Gramoll CC BY NC-SA 4.0]

```
#| standalone: true
#| viewerHeight: 600
#| components: [viewer]

from shiny import App, render, ui, reactive
import random
import asyncio
import io
import math
import string
from datetime import datetime
from pathlib import Path

def generate_random_letters(length):
    # Generate a random string of letters of specified length
```

```
    return ''.join(random.choice(string.ascii_lowercase) for _ in range(length))

problem_ID="156"
F=reactive.Value("__")
h=reactive.Value("__")
θ=reactive.Value("__")

attempts=["Timestamp,Attempt,Answer,Feedback\n"]

app_ui = ui.page_fluid(
    #ui.markdown("**Please enter your ID number from your instructor and click to generate yo
    #ui.input_text("ID","", placeholder="Enter ID Number Here"),
    #ui.input_action_button("generate_problem", "Generate Problem", class_="btn-primary"),
    ui.markdown("**Problem Statement**"),
    ui.output_ui("ui_problem_statement"),
    ui.input_text("answer","Your Answer in units of psi", placeholder="Please enter your answ
    ui.input_action_button("submit", "Check Answer", class_="btn-primary"),
    #ui.download_button("download", "Download File to Submit", class_="btn-success"),
)


def server(input, output, session):
    # Initialize a counter for attempts
    attempt_counter = reactive.Value(0)

    @output
    @render.ui
    def ui_problem_statement():
        randomize_vars()
        return[ui.markdown(f"A 2 inch thick board is cut and then glued back together along a

    #@reactive.Effect
    #@reactive.event(input.generate_problem)
    def randomize_vars():
        random.seed(101)
        F.set(random.randrange(2000, 6000, 100))
        h.set(random.randrange(50, 150, 1)/10)
        θ.set(random.randrange(10, 20, 1))


    @reactive.Effect
    @reactive.event(input.submit)
```

```
    def _():
        attempt_counter.set(attempt_counter() + 1)  # Increment the attempt counter on each s

        instr= (F()/(2*h()))*(math.cos(math.radians(θ()))**2)
        if math.isclose(float(input.answer()), instr, rel_tol=0.01):
            check = "*Correct*"
            correct_indicator = "JL"
        else:
            check = "*Not Correct.*"
            correct_indicator = "JG"

        # Generate random parts for the encoded attempt.
        random_start = generate_random_letters(4)
        random_middle = generate_random_letters(4)
        random_end = generate_random_letters(4)
        #encoded_attempt = f"{random_start}{problem_ID}-{random_middle}{attempt_counter()}{co

        # Store the most recent encoded attempt in a reactive value so it persists across sub
        #session.encoded_attempt = reactive.Value(encoded_attempt)

        # Append the attempt data to the attempts list without the encoded attempt
        attempts.append(f"{datetime.now()}, {attempt_counter()}, {input.answer()}, {check}\n"

        # Show feedback to the user.
        feedback = ui.markdown(f"Your answer of {input.answer()} is {check}.")
        m = ui.modal(
            feedback,
            title="Feedback",
            easy_close=True
        )
        ui.modal_show(m)

    #@render.download(filename=lambda: f"Problem_Log-{problem_ID}-{input.ID()}.csv")
    async def download():
        # Start the CSV with the encoded attempt (without label)
        final_encoded = session.encoded_attempt() if session.encoded_attempt is not None else
        yield f"{final_encoded}\n\n"

        # Write the header for the remaining CSV data once
        yield "Timestamp,Attempt,Answer,Feedback\n"

        # Write the attempts data, ensure that the header from the attempts list is not writt
```

```
        for attempt in attempts[1:]:  # Skip the first element which is the header
            await asyncio.sleep(0.25)  # This delay may not be necessary; adjust as needed
            yield attempt


# App installation
app = App(app_ui, server)
```